Loki III: Reverse Engineering Structural Netlists without a Language Model
How far do provenance, a Bayesian L1 and a graph L2 get on their own?

Peter Jamieson
Electrical and Computer Engineering, Miami University
Oxford, OH, USA
jamiespa@miamioh.edu

Abstract—A synthesis tool turns a circuit description into a netlist and throws information away on the way. Loki III asks how much of the design can be recovered without a language model, from provenance (the link from each netlist object back to the source text), a Bayesian hypothesis flow and graph analysis.

Every claim is a hypothesis with one experiment and one verdict. Nine are USEFUL: the provenance answer key covers all 54,306 gate nodes of 150 designs and the pipeline is naming-neutral over 105,646 renamed identifiers; structures that synthesis already reverse engineered are recovered (hard adders 67 of 67) and calibration cuts the calibration error from 0.650 to 0.217; synthesis destroys style (ceiling 0.50 on the gate floor); big designs are mostly repeated structure; a model does far better with the netlist than without (LLM4).

Six are POSSIBLY GOOD and stated with their fragility: a graph network ahead of a 1-WL baseline (one split), an operator-recovery F1, and a loss audit. Structures that synthesis erased recover weakly, clustering does not find the designer's hierarchy, a word-first detector is not shown to beat DANA, and a full language-model run shows no effect of Loki's hints on top of the netlist. Thirteen hypotheses were DISMISSED and sixteen are INCONCLUSIVE. No area, speed or power is measured.

Index Terms—reverse engineering, netlist, provenance, Bayesian inference, graph neural networks, FPGA CAD

I. Introduction

A designer writes a circuit in a hardware description language (HDL) such as Verilog. At the register-transfer level (RTL) the HDL text says what the registers hold and how they change. A synthesis tool turns that text into gates, the basic logic elements such as AND, OR and NOT, and a netlist, the list of those gates and the wires that join them. Synthesis throws information away. It merges operators, flattens the module hierarchy and leaves no source names on the gates. Reverse engineering a netlist means getting back as much of the RTL structure as possible. Gate-Breaker [1] is an earlier study in which a language model reads a netlist with hidden wire names and writes the RTL; it ends by naming the graph tools that its model needed: clustering to name buses, a classification of signals as datapath or control, and partitioning of the netlist.

Loki III (https://github.com/drpaj12/loki) asks how much of the RTL can be recovered without a language model. It stands on provenance: the link from each netlist object back to the HDL that produced it. Odin III, the synthesis front end of the Yggdrasil III suite, keeps that link through elaboration, optimisation and mapping. The link yields an answer key (also called ground truth): what the tool recorded about which netlist objects form which HDL construct. The answer key is what makes recovery measurable, because a recovered structure can be compared with a structure the tool knew.

We write every claim as a hypothesis: a claim that an experiment can confirm or fail to confirm. For each one the paper gives the experiment, the result and one verdict. USEFUL means that the experiment confirms the hypothesis under its own decision rule. POSSIBLY GOOD means that it confirms it only narrowly or fragilely, so the paper states the fragility and never uses such a result alone as evidence for an innovation. INCONCLUSIVE means that the numbers do not decide; we never invent a threshold to force a verdict. DISMISSED means that the experiment does not confirm it. Ideas whose experiments prove their value are the innovations of the main paper; dismissed ideas are one line each in the research log (Appendix A).

Loki has three layers: L0, labelled and anonymised netlist datasets with the answer key; L1, a Bayesian flow that proposes structure hypotheses and settles them with probabilities and equivalence checks; and L2, graph analysis. The innovations that the experiments confirm are these three, in plain English, each with the hypotheses that confirmed it.

1) A benchmark in which the answer is known and the names carry nothing. We build the netlists with a front end that remembers which source text made each gate, give every object a neutral name, and show that scrambling all names changes no

answer (naming neutrality: 476 byte-identical run pairs). The answer key covers every gate node of the 150 designs that build (module coverage 1.000). Confirmed: L0a, L0b.

2) A hypothesis flow that is honest about what it knows. The tool proposes "this group of gates is an adder, a register or a mux", gives each claim a probability learned from training designs, and uses an equivalence check to settle the claims it can. Structures that synthesis already reverse engineered are recovered almost perfectly, and the tool's stated probabilities are far closer to the observed rates after fitting. Confirmed: L1h, L1c.
3) A limit on what any recovery can do, a synthetic test bed, and a count of repeated structure. A set of twin designs measures how much coding style synthesis destroys (H14a); a generated set of idiom designs builds at every mapping level of the flow (H12a); and the large designs of the VTR suite are mostly structure that repeats (H13a), with identical neighbourhoods inside the repeats (H13c). Confirmed: H14a, H12a, H13a, H13c.

Candidate innovations. Three more ideas have only POSSIBLY GOOD support: each experiment leans the right way but is narrow or fragile, so none is a contribution of this paper. (i) Graph analysis with a network. Treat the netlist as a graph and train a graph network to label each gate's role (L2b: a score of 0.4241 against 0.2717 for a baseline that sees only the neighbourhood of each gate, one run on one split of 13 designs). What would confirm it: several random splits of the designs with an interval over designs, and a second benchmark suite. (ii) A score that sees recovery. Comparing recovered Verilog text with the reference cannot tell whether the right adders were found; counting the operator types recovered can (R2-L7: small margins on few pairs, and the flow with fitted probabilities recovers no operator). What would confirm it: a set with many more tie pairs and a flow that recovers operators. (iii) A loss audit. When synthesis merges the logic of two source statements, the construct can no longer be pulled apart from its neighbours. Using the provenance the front end already keeps, we record which pass merged what (L15a, L15d: a mux is found at the gate snapshot and lost after the final mapping pass, but the two ground truths differ in size and the footprints are not like for like). What would confirm it: the same audit with equal-sized ground truths at the two snapshots and on the big designs.

The other answers to the paper's question are negative or undecided. Clustering does not find the designer's hierarchy (L2a, DISMISSED); the structures that synthesis erased recover weakly; a pattern finder that looks for multi-bit signals first is not shown to beat a published method; and a full language-model run (137 problems, three draws) shows that Loki's hints add nothing measurable to the netlist.

Loki's measures are recovery measures (how much of the answer key is found) and not circuit measures. Area, speed and power are not measured by any experiment in this paper, and we do not claim they correlate with the recovery scores. Mímir III, the suite's circuit knowledge base, is where an answer key of this kind would be stored and queried across designs; Loki does not use it yet.

Figure 1 places Loki III in the suite. Yggdrasil III is drawn as a tree. Odin III is the trunk, and its one provenance-tracked IR is what Loki III reads and what supplies its answer key. Loki III is a side branch, a tool and experiment for reverse engineering, next to Leman Russ III (hardware Trojan detection tools), The Norns III (placement with provenance) and Verdandi III (the Micro through the Macro). It runs as a step of Wyrd III, the experiment runner in the canopy. Mímir III, the circuit knowledge base, hangs from the canopy as the fruit. Three wells at the deep roots hold the outside work the suite draws on, including the benchmarks that this paper uses.

II. Background

A. Glossary

The terms of this paper are defined here, in the order in which they depend on one another. Basic FPGA and CAD terms follow the thesis of Jamieson [2] (Chapter 2) and the thesis of Luu [3] (Sections 2.1.1 and 4.1.2). Terms that the Introduction needs (from HDL to the three layers) are also defined where it first uses them.

HDL. Hardware description language: the language, such as Verilog, in which a designer writes a circuit [2].

RTL. Register-transfer level: the HDL text that says what the registers hold and how they change [2].

gate. A basic logic element (AND, OR, NOT); the elaborated design is first written with such primitive gates [2].

netlist. The list of the basic blocks of a design and the nets (wires) that join them; every net is driven by exactly one block [3].

flip-flop. A one-bit storage element that keeps its value between clock edges.

LUT. Look-up table: a small memory that implements any logic function of K inputs ($K = 6$ here). In most FPGAs a LUT and a flip-flop form one basic logic element [2] [3].

technology mapping. The CAD step that covers the logic of a netlist with the cells the target offers: gates for a gate library, LUTs and flip-flops for an FPGA [2].

provenance. The link from each netlist object back to the HDL that produced it.

Odin III. The provenance-tracked synthesis front end under Loki; it keeps that link in one intermediate representation.

answer key / ground truth. What the synthesis tool recorded about which netlist objects form which HDL construct; the target that recovery is scored against.

Gate-Breaker. An earlier study in which a language model rewrites gate netlists into RTL; Loki's method baseline.

hypothesis. A claim that an experiment can confirm or fail to confirm. In L1 a structure hypothesis is the claim that a set of netlist cells is a structure of some kind (register, adder, mux, ...).

L0 / L1 / L2. Loki's three layers: L0 labelled datasets, L1 a Bayesian flow over structure hypotheses, L2 graph analysis.

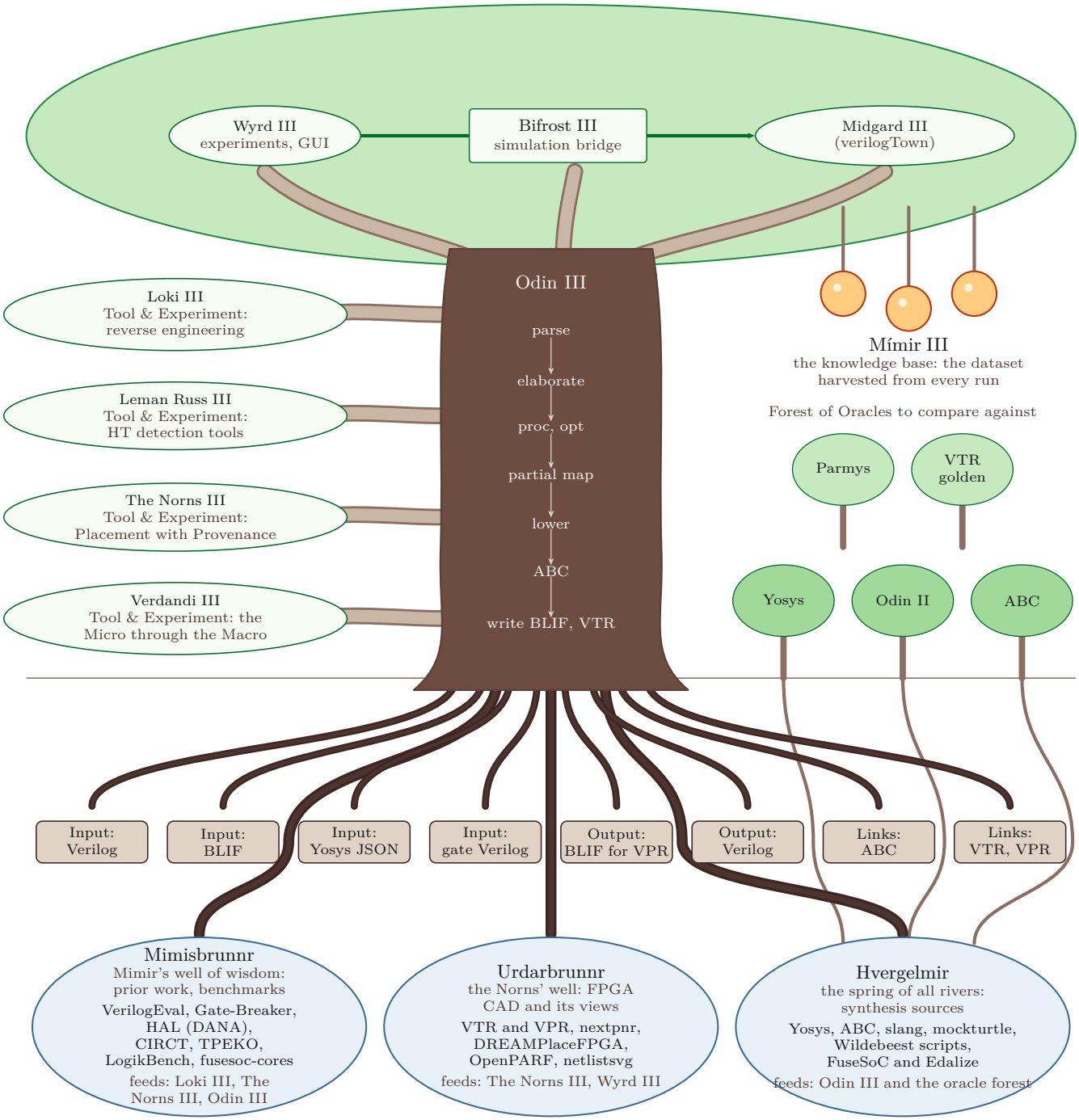


Fig. 1. Yggdrasil III drawn as a tree, and where Loki III sits in it. The trunk is Odin III (https://github.com/drpaj12/odin3); the roots are its inputs and the tools it links to, and three wells hold outside work. The oracles stand beside the trunk. The canopy holds Wyrd III (https://github.com/drpaj12/wyrd), Bifrost III and Midgard III, and the fruit, Mímir III (https://github.com/drpaj12/mimirIII). Side branches carry tools and experiments: Loki III (https://github.com/drpaj12/loki, this paper), Leman Russ III, The Norns III (https://github.com/drpaj12/nornsIII) and Verdandi III (https://github.com/drpaj12/verdandiIII). The hub is https://github.com/drpaj12/YggdrasilIII.

quartile. One quarter of the benchmark designs ordered by Gate-Breaker's complexity score (Q1 simplest, Q4 most complex).
AST / CFG similarity. How alike two RTL texts are by subtree overlap of the syntax trees (AST) and by a kernel on the control-flow graphs (CFG); Gate-Breaker's measures of a recovered text against its reference.
micro designs. The 68 small synthesizable RTL designs (counters, registers, arithmetic, ALUs, muxes, comparators, small state machines) used as the first benchmark.
VerilogEval references. The 156 reference RTL designs of the VerilogEval spec-to-RTL set; the second benchmark.
floor. The abstraction level and technology mapping at which a design is cut and written as the netlist to reconstruct (f1-k6, f1-earch, f2, f3).
renamed-instance protocol. Rename every identifier through one shared permutation, run the pipeline, map the tokens back: equal results show that the pipeline does not use names.
detector. A plug-in that proposes structure hypotheses for a netlist and reports features of them, never a likelihood.
calibration. Fitting priors and likelihood ratios on a training split so that predicted probabilities match observed rates.
ECE. Expected calibration error: the average gap between predicted probability and observed match rate over ten bins.
prover. An engine (cone check, truth table, transfer) that proves a structure hypothesis equal to the netlist; a proof makes it near certain.
count / match (scoring). A hypothesis is counted when it is committed or has probability at least 0.5; it matches an answer-key instance by an overlap rule (Section IV-A).
precision / recall / F1. Precision is the share of counted hypotheses that match; recall is the share of answer-key instances matched; F1 is their harmonic mean.
macro-F1 / micro-F1. F1 averaged over the kinds of structure (macro) or pooled over all instances (micro).
Louvain / modularity / NMI. Louvain is a community-detection method; modularity scores how dense the clusters are; NMI scores the agreement of a clustering with the known module hierarchy.
GNN and 1-WL baseline. A graph neural network (GNN) labels each netlist node from its neighbours; the 1-WL baseline is colour refinement, whose distinguishing power a message-passing GNN cannot exceed.
word. A group of cells that act as one multi-bit signal (a register, a datapath operator, a bus).
operator type-F1. F1 of the recovered word-level operator types (adder, mux, comparator, ...) against the answer key's operators.
fragment. A wrong-looking candidate that is in fact part of a real construct.
word-first detector. A detector that first groups flip-flops into register words by shared clock, enable and reset, then builds the rest around them.
DANA (through HAL). A published method, run through the HAL framework, that groups flip-flops into register words from their shared control nets and the dataflow between them.
McNemar / Wilcoxon test. An exact paired test on discordant pairs; a paired rank test.
synthetic corpus. A seeded generator's 1200 labelled idiom designs (twelve families) with train, iid and ood splits.
signature dictionary. A table from a cell's cone class and WL colour to the construct it usually is, mined from the synthetic corpus.
D14 provenance map. Odin III's version-2 name map: per object, the source-statement units it blends, and labelled join records.
unit, separable, lost. A unit is the statement-level anchor of a construct; it is separable while some object has exactly that unit set, and lost at the first later snapshot where none has.
random-set null / persistence null. The random-set null draws random unit sets of the real sizes at each pass; the persistence null starts from the objects of the previous snapshot and merges at random until the real object count is reached.
LLM pilot arms A, B, C. Arm A: gate netlist; arm B: netlist plus Loki hints; arm C: interface line only (memorisation control).

B. The provenance problem and Odin III

Synthesis modifies the designer's functional intention: flattening removes hierarchy, optimization merges operators, and mapping covers logic with cells that carry no source names. Linking each netlist object back to the HDL that produced it is the provenance problem [4]. Odin III, the provenance-tracked front end under Loki, keeps that link: every object of its one IR records its source location, and the link survives elaboration, optimization, partial mapping and the linked ABC flow. Loki stands on that link: its L0 datasets carry the answer key the synthesis tool kept, which is what makes the recovery statistics in this paper possible at all.

C. Gate-Breaker as the method baseline

Gate-Breaker measures recovery by functional correctness and by the similarity of the recovered and reference RTL: Success Rate (the fraction of K samples that simulate correctly), Success@k (the fraction of designs solved within the first k samples), AST similarity (mean subtree Jaccard over normalized identifiers, depth $d = 3$) and CFG similarity (a normalized shortest-path node kernel over control-flow graphs). Its complexity score weights sequential blocks, generates, loops, instantiations, always blocks, buses, multiplexers, cases, arithmetic operators, assignments and concatenations into quartiles Q1 (simple combinational) through Q4 (complex sequential). Loki reports its own statistics on the same axes where they apply – recovery rate per structure kind, calibration error, clustering quality against the known hierarchy, and GNN accuracy against a 1-WL baseline – and the Gate-Breaker metrics on the replication runs.

D. Dataflow analysis and HAL as a second baseline

HAL, the Hardware Analyzer of the Embedded Security Group [5], [6], is an open framework (MIT licence) for the reverse engineering of gate-level netlists. Its dataflow plugin implements DANA [7], which groups flip-flops into register words from their shared control nets (clock, enable, set, reset) and from common predecessors and successors in the flip-flop dataflow graph, without reading the logic between them. DANA is therefore a word-finding method that is independent of the cell library once the flip-flops are known, which makes it the natural external baseline for the register words that Loki's word-first detector (Section V-F) targets. Section V-F runs both

on the same netlists and scores both against the same Odin III answer key. We credit HAL and DANA in the README and the bibliography; no HAL code is part of this repository, only our gate-library files, a converter and the scorer.

III. The innovations

The Introduction describes each innovation in plain English. This section describes them technically: the floors and the L0 datasets (including the renamed-instance protocol) and the L1 hypothesis flow, which are confirmed innovations, and L2 graph analysis, the operator-recovery score and the loss audit, which are candidate innovations (POSSIBLY GOOD, not contributions). The experiment that confirms or fails to confirm each one is in Section V.

A. Floors: the abstraction level of the netlist

A floor is the abstraction level and technology mapping at which an Odin III design is cut and written out as the L0 netlist, the object the reconstruction receives. Every floor starts from the same elaborated design, the common front proc; fsm; opt; flatten; opt -full; wreduce (FRONT in loki/odin.py), and then maps it with a floor-specific script (floor_script in the same file). Table I lists the floors. Four exist: f1-k6 and f1-earch (one floor per FPGA architecture file; the directory name is f1- plus the architecture's lower-case prefix), f2, and f3.

What a floor's netlist contains decides what is left to reconstruct. At f1 the flow's hardmap step has already recognised adders, multipliers and memories, so they appear as typed hard cells; the construct they implement is part of the netlist, and the labels still name it. At f2 the same logic is only LUT truth tables: an adder is a carry chain of LUTs that has to be found again. At f3 the logic is a network of two-input gates, the form Gate-Breaker's language model receives. The f2 output does not depend on the architecture file, and f3 reads none, so their files are byte-identical across architectures.

Loki uses several floors because what a synthesis flow erases depends on the mapping, and recovery has to be measured against each. f1 keeps the structures the flow already reverse engineered, which is what lets the hard detector of L1 read them from Odin III's unmap facts; f2 removes that crutch with the same source and the same answer key; f3 makes the input comparable to Gate-Breaker's and is the floor on which the AST and CFG similarities are computed (Section V-B). Within one design the floors share one source of truth, the provenance spans of the same elaborated IR, so a difference between floors is a difference of the mapping.

B. L0: provenance ground-truth datasets

In plain English. The datasets are the test bed: netlists built from Verilog by a front end that remembers which source text made each gate, with every name hidden, so that the tool under test cannot cheat by reading names and its answers can be checked against the front end's record.

Technically. The first benchmark set is the 68-design micro suite (counters, registers, arithmetic, ALUs, muxes, comparators, small FSMs) that Odin III already compiles and checks against Yosys; the second, the 156 VerilogEval references, is described in Section IV-D. L0 compiles each design on every floor of Table I. Every netlist object carries its provenance (source file and line), a neutral nk name from one shared counter, and a label: the structure kind (construct_type) is the answer key in labels.csv, and the netlist-visible hard-block kind (cell_type) travels in the graph labels, so structures that are already reverse engineered stay visible to the reconstruction. loki check validates every dataset: 68 designs, 269 floors, a label row for every node, zero errors. The two hard-block designs the v1.0 front end refused now compile on the v1.1.0 SystemVerilog front end, so the set is complete.

Renamed instances. A separate protocol asks the naming question directly: rename every identifier of every netlist (105,646 identifiers across 269 netlists) through a shared permutation, run the pipeline on the renamed floors, and inverse-map every token back. The verifier reports token-for-token equality with the original, zero failures, and the base and renamed score digests are byte-identical (sha256 over all run scores, 204 of 204 pairs) – the pipeline is naming-neutral, so nothing it recovers is an artefact of a name.

C. L1: the Bayesian hypothesis flow

In plain English. The flow proposes "this group of gates is an adder, a register or a mux", gives each claim a probability learned from training designs, and settles the claims it can by an equivalence check.

Technically. L1 is a round-based hypothesis flow over one netlist, with no language model. It reads a floor netlist into a fresh Odin III design (level 0, never edited) in which every cell is a function, and keeps a working copy that grows one level of hierarchy per committing round.

Hypotheses and detectors. A hypothesis is a claim that a set of level-0 cells is a structure of some kind, for example: a flip-flop with enable or synchronous reset (ff_en, ff_srst), a register, a bus of isomorphic slices, an adder, subtractor, comparator or constant comparison (eq_const), a mux or mux_tree, a counter, shift_reg, lfsr or fsm, a datapath or control unit, a per-cell role, or a hard block (hard_add, hard_mul, hard_mem). Nine detector plugins (arith, bus, ff_pattern, fsm, hard, mux, role, seqfn, unit) propose them. A detector reports features as bin labels (a width of 5 to 8, whether an enable function is exact, the NPN class of a cell) and never a likelihood; the likelihoods come from calibration.

Bayesian inference. Each hypothesis h of kind k from detector d carries a log-odds $L(h)$, the logarithm of the odds that it is a real structure, and $p(h) = 1/(1 + e^{-L(h)})$ is its probability. L is a naive-Bayes sum: a prior plus

TABLE I
The floors of L0. All floors run the common front first; the mapping script is the second part of floor_script. Node kinds are those of graph.json (pi, po, const, lut, ff, hard, gate).

Floor	Reads	Mapping script	Netlist contents
f1-k6	read_arch k6_frac_N10_ frac_chain_mem32K_40nm	hardmap; lower; abc –lut 6	LUT6, flip-flops, and hard adder, multiplier and RAM cells that keep their Odin III type
f1-earch	read_arch of the EArch file	same as f1-k6	same kinds, EArch hard blocks
f2	read_arch (cell types only)	lower; abc –lut 6	LUT6 and flip-flops only; arithmetic is soft, so no hard node exists
f3	read_techlib lib/cmos6.o3lib	lower; abc –gates cmos6; dfflibmap cmos6	library gates (BUF, INV, NAND2, NOR2) and DFF flip-flops

evidence items, each the log-likelihood ratio (LLR) of an observation,

$$L(h) = \pi_{d,k} + \mathrm{clip}_{\pm 6.91}\Big(\sum_f w_{\phi(f)}\, \lambda_{k,f}(b_f) + \min\big(1.10\, n_{\mathrm{conf}},\, 3.30\big) - 2.30\, n_{\mathrm{ref}} \Big) + e_h, \quad (1)$$

where $\pi_{d,k}$ is the prior log-odds of that detector and kind; $\lambda_{k,f}(b_f)$ is the LLR of feature f in bin b_f; $w_{\phi(f)}$ weights the feature's family (structural, WL colour, NPN class or L2; the f3 calibration uses the first three only); n_{conf} and n_{ref} are the numbers of confirmed (or committed) and refuted child hypotheses (a bus slice, a register's flip-flops), which raise and lower their parent; and e_h collects fixed records (a sampled check adds $\ln 100 = 4.61$). The clip at $\pm \ln 1000 = 6.91$ limits the double counting that a naive-Bayes sum of correlated features would otherwise allow. A proof sets $L(h) \geq \ln 9999 = 9.21$ (the odds become 9999 to 1), and a counterexample sets $L(h) = -\infty$.

All three tables are calibrated from the training split of L0 (loki calibrate). The flow runs with default tables on every training design; each hypothesis is labelled true when it matches an L0 construct (Section IV-A). The prior is the Laplace-smoothed precision of the detector, $\pi_{d,k} = \mathrm{logit}\, \frac{tp+1}{n+2}$ over n proposals of which tp are true. For example, the f3 comparator detector made 31,036 proposals of which 2 were true, so $\pi = \ln \frac{3}{31035} = -9.24$; the committed value is -9.244259 (experiments/f3-recovery/calib/f3.json). A bin's LLR is $\lambda = \ln \frac{(pos+1)/(POS+B)}{(neg+1)/(NEG+B)}$, with pos and neg the true and false proposals in the bin, POS and NEG the totals of the kind, and B the number of bins. The family weights w are the grid values in $\{0, 0.1, \ldots, 1\}$ that maximise the Bernoulli log-likelihood of the labels, fitted by three sweeps of coordinate ascent on out-of-fold rows (two folds, split by design). Without calibration (–calib none) the priors are 0 and the LLRs are the defaults of bayes.py. The reliability check is the expected calibration error over ten equal-width bins of the predicted probability, $\mathrm{ECE} = \sum_b \frac{n_b}{N} |\bar{p}_b - r_b|$ with r_b the observed match rate.

Proof engines. A hypothesis with a functional claim ("these 9 LUTs compute $A + B$ at width 5") can be proved. Three engines prove claims: cone_check first when the claim has a template, the truth table otherwise or when the cone cannot be bound to the template, and transfer after an exact proof. (i) cone_check, an Odin III pass, proves that the claim's cone of level-0 cells is equivalent to a template word cell (for example \$add, \$sub, \$eq, \$mux) with its ports bound to the cone's boundary nets, under a timeout; the reference functions of the claims (claims.py) cover addition, subtraction, the six comparisons, constant comparison, 2:1 and N:1 multiplexers, counters, shift registers and LFSRs. (ii) The truth-table prover enumerates the claim's inputs and side nets when together they number at most 16, an exact proof or a counterexample, and otherwise simulates 4096 seeded random vectors, which can refute a claim but only raises it by $\ln 100$. (iii) Proof transfer groups open hypotheses by operation, parameters and the 1-WL colour of their output drivers; when one member is proved exactly, the others are tried by a lockstep cone_match of the two cones, and one transferred member per group is proved for real as a spot check, revoking the transfers of the group if it fails. A sequential claim is proved as a next-state function (state in, next state out); an FSM gets no proof.

Deciding the structure. Hypotheses that claim the same cells compete. Two hypotheses conflict when both kinds are exclusive, their cell sets intersect, and neither is a strict subset of the other (a strict subset is nesting: the smaller becomes a child of the larger). For each connected component C of the conflict graph the competitive probability is

$$p_{\mathrm{comp}}(h) = \frac{e^{L(h)}}{1 + \sum_{j \in C} e^{L(j)}}, \quad (2)$$

where the "1" is the option that none of them is real; a hypothesis without conflicts has $p_{\mathrm{comp}} = p$. Two confirmed hypotheses with equal cells are two true forms of the same logic and count as one member of C; the one whose kind ranks higher (a counter above an adder, for example) wins at commit and the other is kept as an alternative. A proof is spent on hypotheses with $p_{\mathrm{comp}} \geq 0.2$, most probable first, at most 200 per round; a hypothesis below 0.01 is rejected, and re-enters when later evidence lifts it. A hypothesis is selected for commit when $p_{\mathrm{comp}} \geq 0.9$, every child that could commit is already committed, and it does not conflict with a committed piece. Selection is greedy in the order (p_{comp} descending, kind rank, cell

Algorithm 1 The L1 round loop (loki/re/loop.py).

```
level_0 ← read netlist as functions; work ← level_0
if floor f1 with hard cells then run unmap on work
for r = 1 to 8 do
    propose: every detector adds hypotheses over the view of work
    score: L(h) by Eq. 1; compete: p_comp by Eq. 2
    prove claims with p_comp ≥ 0.2: cone_check, truth table, transfer
    re-score and compete; reject p_comp < 0.01
    select p_comp ≥ 0.9 (children first); commit by raise and submod
    if nothing committed, nothing new, |ΔL| ≤ 10^−6 then
        break
    end if
end for
write design.v, overlay.json, report; equiv-check against the input
```

count descending, id), and the losers of a conflict become rejected or alternative.

Commit and the final structure. A selected hypothesis is rebuilt on a copy of the working design: a hypothesis is raised to a single word cell only if its kind is one of adder, subtractor, comparator or multiplexer, its claim has a cone_check template, and it is proved by cone_check (directly or by transfer); the cone is re-checked by cone_check unconditionally before the raise. A claim proved only by truth table is wrapped, not raised, and counters, shift registers, LFSRs, FSMs, registers, constant comparisons, multiplexer trees and carry-in adders are never raised. Every selected piece except buses and roles (which only set attributes) is wrapped by submod in a module named for its kind, whose instance is tagged with the hypothesis id, kind, confidence p_{comp}, proof status and the recovered function. If raise or submod fails, the offending hypothesis is rejected with a record of the error and the commit is retried once without it (reject_failed). The structure that L1 reports is therefore exactly the set of committed pieces in design.v; the alternatives, rejected and open hypotheses stay in overlay.json with their evidence. Algorithm 1 summarises the loop. It stops when a round commits nothing, proposes nothing new and changes no log-odds by more than 10^{-6} (a fixed point), after eight rounds, or at the time budget, then flattens and lowers the final design and checks it against the input with equiv-check; a result of differs is a bug and the run exits with an error.

The hard detector reads the unmap pass's facts: a hard adder, multiplier or memory that the flow already reverse engineered is recovered directly, with the decline reasons the pass recorded (carry-in chains and the like) kept as open hypotheses. The hard detector does not run on f2 or f3.

D. L2: graph analysis

L2 reads the L0 graph.json files and nothing else: never key.json and never the BLIF. It has three parts.

Structural features. Each node gets numeric features (fan-in, fan-out, logic level, LUT width, hard-port count), its kind and, on hard cells, its cell_type, plus three families: colours of 1-WL colour refinement (a hash of the node's label and the multiset of its neighbours' colours, per round, so that equal colours mean equal unfolded neighbourhoods), the NPN class of a LUT's function (invariant under permutation and negation of the inputs and the output), and the ORCA orbit counts of the 4-node graphlets around the node. Before graphlets are counted, the global signals (clock, reset, constants, global enables) are classified by role and their edges removed, so that a clock does not connect everything.

Clustering. Four methods partition a netlist graph into candidate blocks: k-means on the feature space ($k = \text{clip}(\text{round}\sqrt{N/2}, 2, 200)$), affinity propagation on a sample of at most 3000 nodes with the remaining nodes assigned to the nearest exemplar, spectral clustering (a spectral embedding of the largest connected component, then k-means), and Louvain community detection (resolution 1, no k). The first two use node features; the last two use the graph structure. A partition is scored by its modularity on the undirected simple projection of the graph, by its NMI against the known module_path and construct_type labels and by its ARI against module_path only over the LUT, flip-flop, hard and gate nodes. These two numbers are a quality report, never an input to a method: modularity says whether the clusters are dense subgraphs, NMI whether they are the designer's modules.

GNN against 1-WL. A node classifier predicts each node's construct_type: a two-layer GraphSAGE (GCN as the variant) of hidden width 64 with ReLU and dropout 0.2 over the graph and its reverse edges, trained with Adam (learning rate 0.01, weight decay $5 \cdot 10^{-4}$) for a fixed 100 epochs, full-batch, with class weights $1/\sqrt{\text{count}}$. The split is L0's, by design, with the class vocabulary taken from the training designs only (rarer values, and values first seen in a test design, are one class ___other___); there is no validation set and no early stopping, so nothing is tuned on a test design. Loss and metrics cover only the LUT, flip-flop, hard and gate nodes with a known target. The question is whether message passing earns anything over what the colour refinement already knows, because a message-passing GNN is at most as discriminating as 1-WL [8]. So the 1-WL baseline predicts, for each node, the most common training class of its WL colour (backing off from round 4 to round 0 when a colour has fewer than five training nodes), and the GNN has beaten it only if its test macro-F1 is higher. Two further baselines are reported on the same nodes, the majority class and a logistic regression on the features alone. The task is that of GNN-RE [9], which maps each gate of a flat netlist to the module it belongs to; Loki's model is a baseline for the same task, not a re-implementation, and the numbers are not compared with that paper's.

E. Operator type-F1 and the loss audit

In plain English. Comparing recovered Verilog text with the reference cannot tell whether the right adders were found, so we also count the operator types that were recovered. The loss audit asks a different question: when synthesis merges the logic of two source statements, the construct can no longer be pulled apart, and the audit records which pass did the merging, using the provenance map that the front end already keeps.

Technically. The L0 key lists the word cells of the elaborated reference. A run's recovered operators are its counted hypotheses, mapped to an operator class and an operand width (add, sub, mul, eq, ne, lt, le, gt, ge, mux, register). Precision and recall are multiset overlaps, so the score does not depend on which cells carry an operator; type ignores the width and tw requires it. The loss audit reads the D14 provenance map after each pass of the Odin III flow (proc, fsm, opt, flatten, opt -full, wreduce, lower, abc) and tracks units: a unit is separable at a snapshot when some object holds exactly that unit set, and lost at the first later snapshot where none does (Section V-G gives the protocol).

IV. Methodology

The setup is: the floors and the two benchmark sets (Section IV-D), the scoring of L1 and L2 against the answer key (Section IV-A), the metrics and the statistics (below), and the language-model baseline of Gate-Breaker. All experiments except the language-model pilot and the full run use no language model.

A. Evaluation protocol: how L1 and L2 are scored

Gate-Breaker evaluates a stochastic model: for each design the model is sampled K times, each sample's RTL is extracted and simulated against the VerilogEval testbench, and the results are aggregated per complexity quartile as the success rate (the fraction of the K samples that pass), Success@k (the fraction of designs with a passing sample among the first k), and the AST and CFG similarity of the recovered RTL to the reference. Loki's L1 and L2 are deterministic and their output is a structure, not an RTL text, so the same four questions are asked in different terms. Table II sets the two side by side.

L1, step by step.

1) Split. L0 assigns each design to train or test by a hash of the design name (sha256(seed:design) mod $100 < 80$ is train), so the split is fixed by the name and independent of the run.
2) Calibrate on the training designs of one floor (loki calibrate): the priors, bin LLRs, family weights and reliability curve of Section III-C.
3) Run loki re on every test netlist, calibrated and, as a diagnostic, with –calib none. The run never reads the answer key.
4) Ground truth. From words.json, the L0 word cells give ground-truth instances (type, F_g) with F_g the set of final cells of the word; counters, mux trees and FSMs are derived from the word connectivity.
5) Count and match. A hypothesis counts when it is committed or its $p_{\text{comp}} \geq 0.5$ (–score-p); it matches a ground-truth instance of an accepted type when the Jaccard of their cell sets is at least 0.5, greedily one-to-one by Jaccard. Precision is matched over counted, recall matched over ground truth, and average precision sweeps the threshold from 0.1 to 0.9.
6) Buses, roles and hierarchy. A bus matches a ground-truth bus (nodes sharing a module path and bus name, at least two bits) at the same Jaccard; its bit order is scored exactly and up to reversal. Roles are scored as precision and recall of the predicted control and datapath cells. The hierarchy similarity compares the clusters of the module paths to the committed pieces with the Dice score $F(g,h) = 2|g \cap h|/(|g| + |h|)$, $\text{HS}_{gt} = \sum_g |g| \max_h F(g,h) / \sum_g |g|$ and its converse HS_{l1}, and reports their harmonic mean.
7) Aggregate. loki score writes one score.json per run and loki score-all the floor-by-kind tables, the reliability curve of p_{comp} with its ECE, and, with –compare, the ablation against –features none.
8) Naming neutrality. The renamed twin of the dataset is run and scored against the original-named key; the base and renamed score.json must be byte-identical.

Of the Gate-Breaker metrics, the AST and CFG similarities are used on f3 as in the paper: AST similarity is the mean, over the subtrees of the reference, of the best Jaccard similarity to a subtree of the candidate (depth 3, node kinds normalised); Gate-Breaker computes both on functionally correct samples only, Loki's are unconditional; CFG similarity is the node kernel over shortest-path histograms of the control-flow graphs. They are computed twice per design, for the gate netlist (L0-stop) and for design.v (L1-stop), and the L1 delta is their difference. The success rate and Success@k need samples; a deterministic flow has one attempt per design, so they are reported only for the language-model replication (Section V-K). Formal final_equiv does not play the role of the testbench: it compares the flattened final design with the input netlist, so it shows that the rewrite preserved the netlist (it is trivial on f3, where L1 commits nothing), not that the design is correct against the reference RTL. Loki has no behavioural pass rate against the reference.

L2, step by step. (1) The same split applies. (2) loki analyze cluster writes a partition per method and design, with its modularity and NMI/ARI against module_path and construct_type; the table reports the mean over designs. (3) loki analyze gnn fits the vocabulary and the model on the training designs and writes metrics.json: accuracy and macro-F1 on the training and test nodes for the GNN, the majority class, the feature-only logistic

TABLE II
The Gate-Breaker evaluation and the Loki evaluation, step by step.

Step	Gate-Breaker	Loki
Input	cmos6 gate netlist of a VerilogEval reference, internal nets renamed n_⟨SHA-1 prefix⟩, ports kept	a floor netlist (f1, f2, f3) with names nk; for the neutrality check a renamed twin
Producer	a language model, K samples per design	the L1 flow, one deterministic run per design (fixed seed); no sampling
Correctness	simulate the extracted RTL against the testbench: success rate, Success@k	equiv-check of the flattened final design against the input netlist: final_equiv (formal; on f1 hard cells are cut points and only soft logic is checked). A self-consistency check of the rewrite, not correctness against the reference RTL; Loki has no behavioural pass rate against the reference
Match to the truth	AST and CFG similarity to the reference RTL	each counted hypothesis against the L0 answer key: Jaccard ≥ 0.5, one-to-one; precision, recall, F1, average precision per kind and floor; hierarchy similarity; AST and CFG of design.v against the reference on f3, unconditionally (Gate-Breaker's are computed on functionally correct samples only)
Aggregate	per complexity quartile	micro-averaged per kind and floor over the test split; AST and CFG per quartile of f3 node count

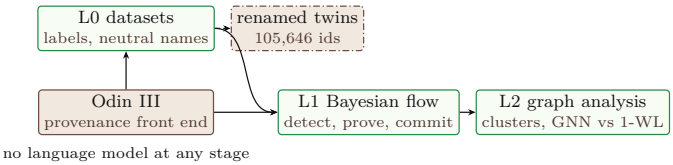


Fig. 2. The Loki III pipeline. L0 compiles the benchmarks through Odin III into provenance-tracked, labelled, anonymised datasets (and their renamed twins). L1 proposes and proves structures in a Bayesian flow that commits into the same IR. L2 reads the graphs back and scores clustering and a GNN against a 1-WL baseline. No stage has a language model.

regression and the 1-WL baseline, and beats_wl. (4) The neutrality check re-runs both on a permuted copy of every identifier.

B. One design through the pipeline

The steps below follow one design, the 4-bit comparator micro bm_DL_4_bit_comparator (three comparison outputs), from source to score. Each step names the file or code that defines it.

1) HDL to Odin III with provenance. read_verilog and the common front elaborate the design. Every IR object keeps its source spans (Node.sources(), the only provenance the binding exposes). Before mapping, Loki snapshots every word-level cell (type, parameters, spans) and every instance site.
2) Map to the floor. lower keeps each word cell's exact span on every gate it creates; abc –lut 6 gives each LUT the union of the spans of its cone. A LUT that merges the three comparators therefore lists all three spans.
3) Join and label. A final cell is matched to a word cell when the word's span set is a subset of the cell's. The primary match gives construct_type (here $eq, $gt, …); a cell with several matches is flagged (n_construct_cells above one) and a cell with none stays unknown: a label is an answer key, never a guess. The module path comes from the instance spans, bus name and bit index from net names and aliases (AeqB[0]), and the datapath or control role from a rule table.
4) Anonymise and write the L0 files. Nets, cells and port bits get names nk from one counter. Loki writes netlist.blif and netlist.v, graph.json (nodes with features and labels, edges), labels.csv (one row per node), words.json (the word-level connectivity the scorer needs) and key.json. Only key.json holds original names; no analysis reads it.
5) Verify. Odin III's equiv-check compares the written BLIF with the floor's own netlist and the anonymised netlist with the original; both must read equivalent before the floor counts as built.
6) L1: normalise. loki re reads netlist.blif (never key.json) into a fresh Odin III design, level 0, in which every logic cell is a function (a LUT by its truth table, a gate by its library function). On f1 with hard cells the unmap pass runs on the working copy and its facts feed the hard detector.
7) L1: hypothesis rounds. Each round runs the detectors, scores every hypothesis (Section III-C), proves the ones worth a proof, resolves conflicts and commits the confident ones (Algorithm 1); the next round sees the committed level.
8) L1: commit and output. A committed hypothesis is rebuilt as a word cell (raise) wrapped in a module (submod) tagged with its kind, confidence and recovered function. At the fixed point Loki writes design.v, overlay.json (every hypothesis with its evidence) and report.json, and checks the final design against the input with equiv-check.
9) L2. loki analyze reads graph.json only: structural features, four clusterings and a GNN (Section III-D).
10) Score. loki score compares the overlay with the answer key (Section IV-A); loki score-all aggregates every run into per-kind tables.

In this micro the f3 comparator, as re-synthesised by the minimal .v front, has 26 logic cells (L1 spec, Section 3); the f2 LUTs carry the comparator spans that the join reads back.

C. Metrics and statistics

Recovery metrics. Precision, recall and F1 (micro and macro) of the counted hypotheses against the answer key; the expected calibration error (ECE); AST and CFG similarity and the operator type-F1 of recovered text against the reference; modularity and NMI of a clustering; the functional pass rate of recovered RTL; and the share of units lost at a pass. All are defined in the Glossary and in Section IV-A.

Area, speed and power. Loki's measures are recovery measures. No experiment of this paper measures area, speed (critical-path delay or near-critical paths) or power. We do not claim that the recovery scores correlate with any of them, and we cite no work that says so; they are not measured here.

Statistics. Paired comparisons over designs use the Wilcoxon signed-rank test; paired pass/fail comparisons use the exact two-sided McNemar test on discordant pairs; intervals are bootstrap intervals (clustered over designs where the designs come in families); the null models (random cell sets of the same size, random merges) are named where they are used. A result is called a difference only when the interval or the exact test shows one; otherwise it is INCONCLUSIVE.

D. The VerilogEval benchmark and its structure

1) VerilogEval spec-to-rtl as the Gate-Breaker benchmark: Gate-Breaker evaluates on VerilogEval [10], taking the spec-to-rtl variant: 156 problems, each a natural-language prompt, a reference solution (*_ref.sv, module RefModule) and a testbench (*_test.sv). The Gate-Breaker flow synthesizes the reference to a cmos6 gate netlist, hands the netlist to the model, and checks the model's RTL against the testbench. Loki uses the same 156 references as L0 sources and reads the reference module as the design; the prompts and testbenches are not inputs to L0, L1 or L2. The references are SystemVerilog, read through Odin III's slang adapter (Section V-A).

2) The complexity score and its quartiles: Gate-Breaker groups the problems by a complexity score (its Eq. 2), a weighted sum of counts of HDL constructs:

$$C = 1000\,c_{\mathrm{Seq}} + 200\,(c_{\mathrm{FG}} + c_{\mathrm{F}}) + 20\,c_{\mathrm{Mo}} + 15\,(c_{@A} + c_{\mathrm{B}}) + 10\,(c_{\mathrm{M}} + c_{\mathrm{Ca}}) + 5\,c_{\mathrm{E}} + 3\,c_{\mathrm{A}} + 2\,c_{\mathrm{C}}. \quad (3)$$

The paper names the terms but does not define them, so the implementation we reuse (complexity.py of the second Gate-Breaker replication) fixes the reading: c_{Seq} counts clocked always blocks (a posedge or negedge in the sensitivity list), c_{FG} for-generate constructs, c_{F} for loops, c_{Mo} module instantiations, $c_{@A}$ all always blocks, c_{B} declarations wider than one bit, c_{M} multiplexers (ternary operators plus non-default case items), c_{Ca} case statements, c_{E} arithmetic operators (+ - * / %), c_{A} continuous assignments and c_{C} concatenations. A clocked block weighs 1000 on its own, so a design in which the counter recognises one scores at least 1000, which is Q3 or Q4.

The quartiles are the paper's absolute bins (its Table II), not rank quartiles of the benchmark: Q1 is 3 to 23, Q2 24 to 111, Q3 112 to 1064 and Q4 1065 to 1326. The 156 references fall 38, 39, 40 and 39 into Q1 to Q4, close to the paper's 39 per quartile. The 150 designs that build on every floor are split 38, 39, 37 and 36: the six missing ones are Prob031, Prob046 and Prob145 (Q3) and Prob078, Prob151 and Prob156 (Q4). Table III gives the counts.

Deviation: pyverilog AST against a regex fallback. The score is computed on the pyverilog AST of the first module when the file parses, and by regular-expression counts of the same terms otherwise, with the method recorded per design. Because the references are SystemVerilog, pyverilog parses only 60 of the 156; the other 96 use the regex counts. The split is skewed by quartile (Table III): 36 of Q1 and 23 of Q2 are AST-scored, but 1 of Q3 and none of Q4. The regex counts operator characters and keywords textually, so they differ from an AST count (the observed Q4 range, 1069 to 3259, exceeds the paper's bin top of 1326 for this reason or because the problems differ; we do not separate the two). The regex also never counts always_comb, always_ff or always_latch blocks, which 34 references use, so c@A and cSeq are undercounted in the 96 regex-scored designs, most of Q3 and Q4; the quartile split and the feature trends below rest on approximate scores. The 156 scores equal those of the replication that computed them first, exactly, so the quartiles here and in Table XXXVII are the same partition.

3) The feature matrix: Gate-Breaker characterises its benchmark by the HDL structures it contains. We build the same kind of matrix for the 150 built designs, from the f3 floor of each, with rules fixed in advance (summarize.py): arithmetic counts the word cells of type $add $sub $mul $div $mod $neg $pos $pow $alu $macc; select/mux those of type $mux $pmux $bmux $demux; storage the register words ($dff, $dffe, $adff, $adffe, $sdff, $sdffe, $sdffce, $dffsr, $aldff, $dlatch, $adlatch); FSM is 1 when Odin III's fsm pass marked any word; control is 1 when any label row has role control or mixed; and wide buses counts the distinct bus names with a bit index of at least 1. The matrix uses one symbol per cell: ✗ for count 0, ✓ for one, ✓✓ for two or more; FSM and control are 0/1 flags, so they show ✗ or ✓ only.

The bus column is buses_wide, not buses. The latter counts every named signal, including single-bit ports, so it is at least 1 for all 150 designs and at least 2 for 147, and would put a check mark in every row. The wide count is the quantity a "multi-bit bus" row means; both columns are committed in features.csv.

Table IV shows 12 designs, three per quartile at fixed

TABLE III
The VerilogEval spec-to-rtl references by Gate-Breaker complexity quartile (absolute bins of Table II; score range observed in the 156 refs; pyverilog AST scores against regex-fallback scores; built = designs with all three floors).

Quartile	Bin	Observed	Refs	AST/regex	Built
Q1	3–23	3–23	38	36/2	38
Q2	24–111	24–90	39	23/16	39
Q3	112–1064	120–1060	40	1/39	37
Q4	1065–1326	1069–3259	39	0/39	36
All			156	60/96	150

ranks by size, so the sample is not hand-picked; Table V counts, per quartile, the designs that show each feature. The storage trend is partly tautological: a clocked block adds 1000 to the score (Eq. 3) and storage counts register words, so storage rises with the quartile by construction; the other trends are conditioned on the score as well. Storage is absent from Q1 and Q2 (0 of 77 designs) and present in 31 of 37 Q3 and all 36 Q4 designs. The other features grow with the quartile: select/mux 8% in Q1 to 56% in Q4, control 3% to 83%, FSMs 0 of 77 below Q3 and 15 of 36 in Q4. Arithmetic is rare throughout (21 of 150, 14%), so the benchmark leans on control, selection and storage more than on arithmetic datapaths. Wide buses are common from Q2 up (90%, 59%, 100%) but nearly absent in Q1 (8%), whose members mostly have only single-bit signals. The FSM flag is Odin III's detection, not a reading of the source: Prob107_fsm1s in Table IV has storage and control cells but no FSM mark.

4) Benchmark size per technology mapping: Table VI gives the size of the 150 designs on each floor. The averages hide a heavy tail. Over all designs the mean node count is 163.5 on f1-k6, 111.4 on f2 and 362.0 on f3, but the medians are 14.0, 13.5 and 25.0: most references are small (125 of 150 have at most 100 nodes at f3) and a few are large (9 exceed 1000; the largest, Prob144_conwaylife, has 15,487). The gate mapping multiplies the node count of the LUT mapping by about 3 on the mean and 1.9 at the median (362.0 against 111.4 on average, 25.0 against 13.5 at the median), as expected when a LUT of up to six inputs becomes a network of cells from a library of one- and two-input gates (BUF, INV, NAND2, NOR2). f1-k6 sits above f2 on average and in Q3 and Q4 (109.4 against 77.4; 453.9 against 269.8) while the two coincide in Q1 and Q2 (6.3 and 6.1; 100.2 and 100.0): the f1-k6 floor keeps hard cells (21 designs hold hard adders) where f2 spreads arithmetic over LUTs, which is consistent with the excess but is not isolated here. By quartile on f3, the mean grows from 9.9 nodes in Q1 to 972.1 in Q4, but the Q4 median is only 51.5: the complexity score tracks the number and kind of HDL constructs, not the netlist size. For scale, the micro suite's f3 floors hold 50,047 nodes over 65 designs, a mean of about 770.

TABLE IV
Benchmark feature matrix, a sample of 12 of the 150 designs (the designs at ranks $\lfloor n/6 \rfloor$, $\lfloor n/2 \rfloor$ and $\lfloor 5n/6 \rfloor$ by f3 node count inside each quartile). ✗: feature absent; ✓: one; ✓✓: more than one; FSM and control are 0/1 flags. Features are read from the f3 floor (Section IV-D3).

Design	Q	f3 nodes	Arith	Sel	Stor	FSM	Ctl	Bus
Prob028_m2014_q4a	Q1	3	✗	✗	✗	✗	✗	✗
Prob036_ringer	Q1	8	✗	✗	✗	✗	✗	✗
Prob091_2012_q2b	Q1	17	✗	✗	✗	✗	✗	✓
Prob125_kmap3	Q2	15	✗	✗	✗	✗	✗	✗
Prob015_vector1	Q2	32	✗	✗	✗	✗	✗	✓✓
Prob123_bugs_addsubz	Q2	166	✓✓	✓	✗	✗	✗	✓✓
Prob107_fsm1s	Q3	11	✗	✓	✓	✗	✓	✗
Prob117_circuit9	Q3	28	✓	✓	✓✓	✗	✓	✓
Prob045_edgedetect2	Q3	89	✗	✗	✓✓	✗	✗	✓✓
Prob128_fsm_ps2	Q4	30	✗	✗	✓	✓	✓	✓✓
Prob140_fsm_hdlc	Q4	52	✗	✗	✓	✓	✓	✓
Prob115_shift18	Q4	1121	✗	✓✓	✓✓	✗	✓	✓✓

V. Experiments

One subsection per hypothesis that is USEFUL or POSSIBLY GOOD, and per INCONCLUSIVE answer that the paper's question needs. Each states the hypothesis, the experiment, the result and the verdict. DISMISSED and the remaining INCONCLUSIVE hypotheses are in the research log (Appendix A). The experiments were run in rounds (0 to 6); a subsection names the round where its text refers to one, and the appendix gives their order.

A. Naming neutrality and the answer key (L0a, L0b)

Hypotheses. L0a: the pipeline uses no names, so renaming every identifier changes no result. L0b: the provenance answer key reconstructs the reference on every floor with no recovery step.

Experiment. For L0a, every identifier of every netlist (105,646 identifiers across 269 netlists) is renamed through one permutation, L1 and L2 are run on the renamed and the base floors, and the tokens are mapped back. For L0b, the answer key is counted on the 156 VerilogEval references.

Result. L0a: all 476 pairs of L1 runs are byte-identical (204, 204 and 68 sets); the L2 clusters are identical on 271 of 272 partitions, and the one difference is a spectral nondeterminism that also occurs between two runs of the base design. The protocol also exposed a rename-script defect (library .blackbox model names were renamed, which silently removed every hard proposal on renamed floors), which was fixed. L0b: all 54,306 gate nodes of the 150 designs that build carry a label, a cell type and an HDL span; module coverage is 1.000 and signal coverage has geometric mean 0.986; six of the 156 references do not build.

Detail of L0b. The experiment is run on the 156 VerilogEval spec-to-rtl reference designs (Section IV-D) next to the micro suite. The experiment builds the three floors f1-k6, f2 and f3 for every reference (f1-earch is not

TABLE V
Designs showing each feature, per complexity quartile: count (percent of the quartile). The last column counts designs with two or more instances of the feature (always 0 for the 0/1 flags). Wide buses are distinct multi-bit buses; a design with only single-bit ports has none.

Feature	Q1 (n=38)	Q2 (n=39)	Q3 (n=37)	Q4 (n=36)	All (n=150)	≥ 2
Arithmetic	2 (5%)	4 (10%)	7 (19%)	8 (22%)	21 (14%)	9
Select/mux	3 (8%)	10 (26%)	12 (32%)	20 (56%)	45 (30%)	12
Storage	0 (0%)	0 (0%)	31 (84%)	36 (100%)	67 (45%)	12
FSM	0 (0%)	0 (0%)	1 (3%)	15 (42%)	16 (11%)	0
Control	1 (3%)	4 (10%)	15 (41%)	30 (83%)	50 (33%)	0
Wide buses	3 (8%)	35 (90%)	22 (59%)	36 (100%)	96 (64%)	55

TABLE VI
Benchmark size per technology-mapping floor, 150 designs: mean / median node count (top), and per complexity quartile (bottom; nodes). Nodes are all graph nodes of the floor netlist: primary inputs and outputs, constants, cells.

All 150 designs	f1-k6	f2	f3
Nodes	163.5 / 14.0	111.4 / 13.5	362.0 / 25.0
Cells	114.6 / 5.0	62.5 / 4.0	313.2 / 13.5
Nets	218.3 / 12.0	93.5 / 10.0	344.2 / 21.0
Nodes by quartile	**f1-k6**	**f2**	**f3**
Q1 (n=38)	6.3 / 5.0	6.1 / 5.0	9.9 / 8.0
Q2 (n=39)	100.2 / 26.0	100.0 / 26.0	238.2 / 32.0
Q3 (n=37)	109.4 / 18.0	77.4 / 13.0	260.6 / 28.0
Q4 (n=36)	453.9 / 26.5	269.8 / 26.5	972.1 / 51.5

TABLE VII
The L0 ceiling on f3, micro suite and VerilogEval references. Both columns are counts over all built designs; the last row is the per-design coverage of declared signals.

	Micro	VerilogEval
Source designs	68	156
Built on f3	65	150
Refused by the reader	3	2
Failed equivalence gate	0	4
Gate nodes, each with a label row	50,047	54,306
Rows with cell_type and HDL span	100%	100%
Module scopes / reference modules	112 / 113	150 / 150
Leaf signal names / declared signals	611 / 666	841 / 745
Per-design signal coverage: median	100%	100%
minimum	55.6%	75.0%
geometric mean	–	0.986

built; one architecture is enough because f2 and f3 do not depend on it).

Coverage of the build. Of the 156 references, 150 build on all three floors, and loki check reports zero errors on the assembled dataset. Two are skipped by Odin III's SystemVerilog reader: Prob151_review2015_fsm and Prob156_review2015_fancytimer (both Q4), with ODIN3_ERR_PARSE ... an explicit 4-bit cast is not supported. Four fail Loki's own equivalence gate and are recorded as failed rather than forced in: Prob031_dff (the reference initialises q to 1'hx; the writer-versus-Odin III BLIF check reports differs on f1 and f2, while f3 was equivalent) and Prob046_dff8p, Prob078_dualedge and Prob145_circuit8, whose negedge flip-flops make equiv-check return an error status for both comparisons. One design, Prob144_conwaylife, was killed by Wyrd's default 2 GB step memory limit (peak 4.3 GB) and built after a rerun with an 8 GB limit. Reading .sv sources needed two fixes in Loki (SystemVerilog is read with –nolatches only, since –vtr-compat is refused for it, and the slang adapter is registered with the process); both are in loki/odin.py and apply to the experiment run.

The ceiling. On f3 the 150 designs have 54,306 gate nodes. Every node has a label row, a cell_type and an HDL span: the label, cell_type and HDL coverage have geometric mean 1.000 over designs. Label coverage is rows over nodes, so it shows that a row exists, not that its fields are informative; unmatched cells stay unknown. The 150 module scopes of the 150 reference modules are all present (module coverage 1.000). Signal coverage, the recovered leaf signal names over the signals the reference declares, has geometric mean 0.986 (arithmetic mean 1.029): 120 designs are at exactly 1, 28 are below it, and 2 are above it, where flattened array elements add names. The micro suite shows the same shape with a slightly lower ceiling (Table VII): 99.1% of module scopes and 91.7% of signal names (an aggregate ratio, where VerilogEval's 841 over 745 is inflated by those two designs; capped at each design's declared count it is 715 over 745, 96.0%). The two sets agree on the message of Section V-B: every node has a label row, a cell_type and an HDL span, and what the mapping erased is the signal names, not the label rows. (Label coverage counts rows, not informative values: a cell that no rule matched keeps the value unknown.)

Caveats, reported with the numbers. (i) –nolatches turns inferred-latch paths into don't-care and lower –async sync turns asynchronous resets into synchronous clears, so a built netlist can differ from the reference's behaviour even though Loki's own checks (netlist against its anonymised twin and against Odin III's writer) pass; the build gate guarantees internal consistency, not equality with the reference. (ii) The reference module, instance and signal counts come from pyverilog where it parses the reference (130 of the 150) and from a regex fallback otherwise (20); the regex counts are approximate and the reference signal count is the weakest number in this section. (iii) The same pyverilog limit decides the complexity quartiles (Section IV-D2). (iv) A train/test split by design is fixed by the manifest rule (117 train, 33

TABLE VIII
L1 recovery on the renamed floors, per kind. Precision/recall/F1 against the L0 ground truth; calibrated (micro-fit) columns. The hard kinds recover from unmap facts; the rest must be re-found from the gates.

Kind (f1)	Precision	Recall	F1
hard_add	0.985	1.000	0.993
hard_mul	1.000	1.000	1.000
ff_en	–	–	0.364
register	–	–	0.021
mux	–	–	0.018
mux_bit	–	–	0.081
macro-F1 (all kinds)			0.354
Kind (f2)			
adder	1.000	0.134	0.237
subtractor	1.000	0.074	–
mux_bit	–	–	0.100
adder_cin	–	–	0.029
macro-F1 (all kinds)			0.108

test on f3); it is not used here, where no model is fitted.

Verdict. L0a USEFUL: naming is neutral, so nothing the pipeline recovers is an artefact of a name. L0b USEFUL: the answer key covers every gate node and every reference module; what the mapping erased is signal names, not label rows.

B. L1: what the flow already reverse engineered, and what it erased (L1h, L1c, L1s)

Hypotheses. L1h: structures that the synthesis flow already reverse engineered (hard adders and multipliers) are recovered almost perfectly. L1c: calibration makes the predicted probabilities honest. L1s: the flow also recovers the structures that synthesis erased.

Experiment. L1 runs on the renamed floors with a calibration fitted on the training designs, and is scored against the answer key (precision, recall, macro-F1, ECE).

Result. Table VIII reports L1 on the renamed floors (identical to base, as the neutrality check requires). The headline result is the hard detector: hard adders recover 67 of 67 with precision 0.985, hard multipliers 30 of 30, because the synthesis flow itself already reverse engineered them and unmap says so. The rest is the honest difficulty: on f1 the calibrated macro-F1 is 0.354 (counted 271, matched 107, ground truth 688), with ff_en at 0.364, registers at 0.021 and muxes at 0.018; on f2 it is 0.108 (91/19/686), with adders at F1 0.237 (precision 1.000, recall 0.134) and most other kinds near zero. Without calibration the flow proposes far more and confirms little: f1 macro-F1 0.202 with 1,128 counted (323 of them declined-adder proposals), f2 0.075. Calibration is honest about itself: ECE 0.217 calibrated against 0.650 uncalibrated.

The gate netlist (f3), hypothesis F3. Gate-Breaker's own input is a cmos6 gate netlist, so we built the f3 floor (65 of 68 micro designs; the gate reader refuses three hard-block sources) and asked whether L1 recovers structure from it.

TABLE IX
Round-2 L1 (RQ-L1): recovery from the f3 gate netlist re-mapped to LUT6 against the native f2 floor, same 65 micro designs. test is the 15 held-out designs, all the 65. Calibration: the micro-fit f2 table (gate rows: the f3 table). Hard kinds excluded from counts and averages; macro-F1 averages the kinds with a counted prediction.

Netlist	Calib.	Split	Runs	Counted	Matched	GT	micro-F1	macro-F1
f3 gate netlist	calibrated	test	15	0	0	116	0.000	0.000
	uncalibrated	test	15	1,166	2	116	0.003	0.038
f3 re-mapped to LUT6	calibrated	test	15	68	2	116	0.022	0.076
	calibrated	all	65	92	14	580	0.042	0.062
	uncalibrated	test	15	252	4	116	0.022	0.044
	uncalibrated	all	65	921	31	580	0.041	0.049
f2 floor (native LUT6)	calibrated	test	15	72	4	116	0.043	0.153
	calibrated	all	65	88	16	579	0.048	0.095
	uncalibrated	test	15	255	13	116	0.070	0.121
	uncalibrated	all	65	930	32	579	0.042	0.070

On the 15 f3 test designs (23,098 gate nodes) no construct is counted (0 of 144 ground-truth instances, macro-F1 0.0) and no hypothesis is committed: the best calibrated confidence in any test design is a bus at $p = 0.764$, below the 0.9 commit threshold. Without calibration the detectors count 1,166 constructs for 2 matches (macro-F1 0.038), and 2,569 counted bus hypotheses match none of the 120 ground-truth buses. The answer key itself is complete at this floor: all 50,047 gate nodes carry a label row, a cmos6 cell type and an HDL span. F3 is DISMISSED; it is the reason for the next experiment.

The similarity probe on f3. Gate-Breaker's AST and CFG similarity (Section II) are the measures on which a recovered text is compared with the reference RTL. With the f3 calibration L1-stop is byte-identical to L0-stop, so both measures have a delta of exactly 0.0000 in every quartile. Uncalibrated, L1 commits 1,535 hypotheses in 11 of the 15 designs and moves the means by +0.0111 (AST) and −0.0075 (CFG) with mixed signs per quartile; the commits are mostly wrong eq_const and bus pieces, so this is not evidence about recovery quality. A flatten control (Odin III proc; flatten, which also lowers the committed processes) inlines the committed hierarchy and raises both measures over the gate netlist: AST by +0.0533 over all 15 designs (0.4576 against 0.4044) and CFG by +0.0559 over the 12 designs the probe can score (0.6941 against 0.6382). Algorithms 2 and 3 of Gate-Breaker read one module, so a recovery that produces hierarchy is penalised until it is flattened: a limit of the measure and not a property of the recovery. The per-kind table and the per-quartile table are in Appendix B.

Hypothesis R2-L1 (round 2): the f3 failure is the detectors' vocabulary and not missing information. The L1 detectors read LUT covers, and the f3 netlist is a cmos6 gate netlist. The hypothesis predicts that re-mapping the same gates into LUT6 recovers close to the f2 rate. We read each f3 netlist.blif into Odin III, run abc –lut 6, and give the result to loki re as an f2 netlist. Odin III drops the cell names in abc, so each of the 8,712 LUTs is expanded back to the gates it covers: a bit-parallel simulation signature matches its output to the f3 net of the same function,

and the cover is the cone of that net down to the matched inputs. 0.230 of the LUTs ABC resynthesised implement a function that is no f3 net; they take the support-contained share of their consumers' covers, and 153 are left with an empty cover. The expanded hypotheses are then scored against the unchanged f3 key, and the native f2 floor of the same 65 designs, with the same calibration, is the comparison (Table IX). On the gate netlist the calibrated flow matches 0 of 116 instances in the 15 test designs. After the LUT6 re-map it matches 2 (native f2: 4), and over all 65 designs 14 against 16 of 580, a micro-F1 ratio of 0.87 in-sample (the 65 designs include the designs the f2 calibration was fitted on) against 0.51 on the held-out test split. The decision rule asked for a ratio of at least 0.8 on both splits, so H1 is inconclusive. The in-sample recovery is that of f2, on the same kinds (adders, subtractors, muxes), but the 15-design test split holds only a handful of matches (2 against 4), so its ratio is noise in either direction. The result is consistent with the f3 failure being the detectors' vocabulary and not missing information in the gates, but does not show it; the f2 level it would recover to is itself low.

Verdicts. L1h USEFUL: hard adders 67 of 67 (precision 0.985) and hard multipliers 30 of 30, because the synthesis flow itself reverse engineered them. L1c USEFUL: ECE 0.217 calibrated against 0.650 uncalibrated. L1s INCONCLUSIVE: the recovery is weak (macro-F1 0.354 on f1, 0.108 on f2) but no threshold or baseline was set, so we do not call it a failure. The LUT6 re-map (round-2 hypothesis R2-L1) is INCONCLUSIVE: it reaches the f2 level in-sample but the held-out split has too few matches to decide.

C. L2: clusters that miss the hierarchy, and a GNN that beats 1-WL (L2a, L2m, L2b)

Hypotheses. L2a: community methods find the designer's hierarchy and centroid methods do not. L2b: a GNN with node, graphlet, WL and NPN features beats the 1-WL baseline. L2m is a measurement, not a hypothesis: the modularity of the four clusterings.

Experiment. Four clusterings of the 68 micro designs are scored against the known module hierarchy; the GNN is trained on the labelled training designs and scored on 13 test designs (3,932 test nodes), once.

Result. Table X scores the four clusterings against the known module hierarchy. The measure of the hierarchy is NMI against the module path of each cell, and it ranks the methods in the order opposite to the hypothesis: k-means 0.3220, affinity propagation 0.2708, spectral 0.1378, Louvain 0.1360. Modularity (L2m) is another matter: Louvain 0.4220 and spectral 0.3620 are well above zero and k-means (-0.2912) and affinity propagation (-0.2264) are negative, but modularity scores how dense the clusters are in the graph, not how well they match the designer's modules, so it is not a hierarchy measure and does not rescue L2a.

TABLE X
L2 clustering against the known module hierarchy (68 designs, mean over the renamed floors, identical to base). Modularity (L2m) and NMI(module_path), the hierarchy measure (L2a), per method.

Method	Modularity	NMI
Louvain	0.4220	0.1360
Spectral	0.3620	0.1378
Affinity propagation	-0.2264	0.2708
k-means	-0.2912	0.3220

271 of 272 partition records are identical between base and renamed floors; the one difference exposed a determinism defect in the spectral path (repeated runs on the same base design give modularities -0.0079, -0.0410 and 0.0252), reported as an open issue rather than hidden.

The GNN result answers the paper's question directly. A node-only GraphSAGE loses to the 1-WL baseline on real micros (macro-F1 0.2413 against 0.2711); combining node, graphlet, WL and NPN features turns it around (0.3545 against 0.2711 on the earlier real-micro run), and the full labelled run of this experiment gives test macro-F1 0.4241 against 0.2717 (accuracy 0.8863 against 0.8492, 3,932 test nodes across 13 designs). The features carry the recovery, not the message passing alone.

Verdicts. L2a DISMISSED: the only hierarchy measure, NMI against the module path, ranks the centroid methods above the community methods (k-means 0.3220 against Louvain 0.1360), the reverse of the hypothesis. L2m MEASUREMENT: Louvain has the highest modularity (0.4220), a statement about graph density and not about the hierarchy. L2b POSSIBLY GOOD: macro-F1 0.4241 against 0.2717 for 1-WL, but it is one run on one split of 13 designs with no interval, and the node-only GNN loses to 1-WL (0.2413 against 0.2711); the claim is that the features carry the recovery, not that the GNN generalises.

D. An instrument that sees recovery, and where the language model stands (R2-L7, R2-L2)

Hypotheses. R2-L7: a word-level operator F1 separates runs that AST and CFG similarity tie on. R2-L2: Loki's AST and CFG similarity is below the language model's on Q1 and Q2, does not stay flat from Q1 to Q4, and Loki's output is formally equal to its input.

Experiment. Round 2 is compute-only: no language-model call was made, and the language-model figures are the round-1 Gate-Breaker replication, read again. Every number in this subsection is a macro generated from the summary tables of experiments/round2. R2-L7 rescored every run set with the operator type-F1 (Section III-E); R2-L2 ran loki re on all 150 VerilogEval netlists and scored the output Verilog against the reference.

Result. R2-L7: are AST and CFG blind to what Loki recovers? Table XI rescores every run set. The floors separate cleanly, which neither AST nor CFG can show because both exist only on f3: type-F1 0.349 on f1 (the

TABLE XI
Operator-recovery F1 (RQ-L7) of the existing and new L1 runs: the multiset of recovered operators (class and operand width) against the operators of the L0 key, pooled over designs. type ignores the width, tw requires it.

Run set	Designs	Key ops	Got ops	P	R	type-F1	tw-F1
f1-k6, calibrated	68	468	169	0.657	0.237	0.349	0.204
f1-k6, none	68	468	964	0.233	0.481	0.314	0.166
f2, calibrated	68	468	21	0.810	0.036	0.070	0.061
f2, none	68	468	756	0.180	0.291	0.222	0.080
f3 gates, calibrated	15	108	0	–	0.000	0.000	0.000
f3 gates, none	15	108	1,161	0.009	0.102	0.017	0.008
f3 as LUT6, calibrated	65	463	19	0.895	0.037	0.071	0.062
f3 as LUT6, none	65	463	753	0.190	0.309	0.235	0.086
VerilogEval f2, calibrated	150	3,585	11	0.727	0.002	0.004	0.002
VerilogEval f2, none	150	3,585	1,151	0.196	0.063	0.095	0.042
VerilogEval f3, calibrated	150	3,585	2	1.000	0.001	0.001	0.001
VerilogEval f3, none	150	3,585	5,488	0.107	0.163	0.129	0.027

TABLE XII
AST and CFG similarity against operator F1 on the 15 f3 test designs (the f3-recovery runs). Calibrated and uncalibrated L1 tie on AST and CFG; the flatten control moves both and leaves the recovery record unchanged.

Condition	AST	CFG	Ops recovered	type-F1
L0-stop gate netlist	0.4044	0.6468	0	0.0000
L1 calibrated	0.4044	0.6468	0	0.0000
L1 uncalibrated	0.4155	0.6394	1,161	0.0173
L1 uncalibrated, flattened	0.4576	0.6941	1,161	0.0173

hard detector), 0.070 on f2 and 0.000 on the f3 gates. On the f3 test designs (Table XII) the calibrated and uncalibrated L1 runs tie on both metrics, AST by 0.0111 and CFG by −0.0075, while operator type-F1 moves by 0.0173; the flatten control raises AST by 0.0422 and CFG by 0.0547 and leaves the recovery record, hence the operator F1, unchanged (0.0000). By the rule H7 is supported by the letter of the rule (weak evidence). The tie the rule was written for, the calibrated flow against the L0-stop netlist (both add nothing to the text), does not separate: their operator type-F1 is 0 against 0. The verdict rests on the calibrated flow against the uncalibrated one, where the calibrated side recovers no operator at all, and on the flatten clause, which holds by construction. Two cautions: the separating level is small (the uncalibrated detectors count mostly wrong constructs), and the flatten comparison holds the record fixed by construction, because the operator F1 reads the recovery record and AST and CFG read the text. The operator F1 does not replace them; it is the measure that sees recovery, where they see only form.

R2-L2: Loki against the language model on the same designs. Round 1 scored the language model on Q1 and Q2 of VerilogEval, and Loki's AST and CFG only on micro designs. We ran loki re on all 150 VerilogEval netlists at f2 and f3, calibrated (a calibration fitted on the 117 train designs of VerilogEval itself) and not, and scored the output Verilog (design.v) against the reference .sv with the round-1 AST and CFG code. 21 references that pyverilog cannot parse have no AST value for any candidate, and 6 f3 outputs exceed the CFG probe's size limit (l2_coverage.csv). On the designs both are scored on (Table XIII), Loki's f3 output has AST similarity 0.763 in Q1 and 0.460 in Q2 (common designs, $n = 36$ and 36) against the model's 0.894 and 0.761, and CFG similarity 0.751 and 0.743 against 0.942 and 0.890; every gap is significant by a paired Wilcoxon test (l2_paired.csv). The calibration was fitted on the train designs and these means cover all 150, so we also give the held-out test split alone (l2_testsplit.csv): on its 16 Q1 and Q2 designs both tools score on, AST is 0.705 for Loki against 0.893 for the model and CFG 0.802 against 0.927, the same direction. (The calibrated f3 flow commits in only 7 designs, so the fit can change few outputs.) Across the four quartiles (Table XIV) the f3 AST over all scored designs ($n = 38$, 38, 32, 21; a different, larger design set than the common rows above) reads 0.754, 0.455, 0.605 and 0.505, so it falls from Q1 to Q4 without being monotone. The 21 references that do not parse fall in Q1 to Q4 as 0, 1, 5 and 15, so Q3 and Q4 are scored on the designs that parse, and the apparent fall may partly reflect which designs those are. The CFG stays flat (0.750 to 0.768). The calibrated f3 flow commits almost nothing, so its output is the L0 netlist; the uncalibrated flow, which does commit, scores lower, not higher.

What the correctness column proves is narrow. Every one of the 150 runs of each set ends final_equiv = equivalent (150 for the calibrated f3 set), which says that the flattened output is formally equal to the input netlist. It does not say the output matches the reference RTL: that netlist itself passes the official testbench in only 128 of the 150 designs (the round-1 self-check; inferred latches and asynchronous resets are the failures), so Loki's output is exactly as correct as its input, no more. The model is checked against the testbench directly, but only its correct samples enter the similarity columns. On operator recovery the model is far ahead: on the common Q1 and Q2 designs its operator type-F1 is 0.465 (functionally correct samples elaborated with the same Odin III front as the key) against 0.081 for Loki's calibrated f2 run and 0.074 uncalibrated (the calibrated f3 run recovers no operator, 0.0, so the comparison uses f2, a different floor from the one AST and CFG use). By the rule, H2 is inconclusive: Loki is below the model on both metrics in both quartiles (yes), the equivalence holds in all runs (yes), and the similarity does not stay flat from Q1 to Q4 (no). The positioning this measures is that the model is readable and recovers the operators of small designs but is unverified, and Loki is verified but partial; neither number alone is the result, and the arm that combines them (a model given the ReverseProvenance record, L8) was not run.

Verdicts. R2-L7 POSSIBLY GOOD: the rule supports it, but the margins are small on few pairs (tie pair ΔF1 0.0173; the flatten control moves AST by 0.0422 and leaves the operator F1 unchanged), and the calibrated flow recovers no operator at all, so we use the operator F1 only as an added instrument beside AST and CFG and

TABLE XIII

Head-to-head on the VerilogEval designs the language model was scored on (Q1 and Q2, functionally correct samples; AST depth 3 and CFG as in Table XXXVII). Loki columns: the output Verilog of the flow at that floor against the reference .sv; L0 is the netlist before L1.

Metric	LLM (round 1)	f3 netlist (L0)	f3 L1 cal.	f3 L1 none	f2 L1 cal.	f2 L1 none
Q1 AST	0.894	0.763	0.763	0.680	0.804	0.720
Q1 CFG	0.942	0.751	0.751	0.565	0.821	0.664
Q2 AST	0.761	0.460	0.460	0.438	0.459	0.448
Q2 CFG	0.890	0.743	0.743	0.679	0.787	0.764
Q1+Q2 AST	0.827	0.611	0.611	0.559	0.632	0.584
Q1+Q2 CFG	0.916	0.747	0.747	0.622	0.804	0.714

TABLE XIV

Loki on all 150 VerilogEval designs by complexity quartile (mean AST and CFG of the output Verilog against the reference; last column: operator type-F1 of the uncalibrated f2 run). Calibrated f3 equals the f3 netlist wherever L1 commits nothing.

Quartile	n	f3 AST	f3 CFG	f3 cal. AST	f3 cal. CFG	f2 cal. AST	f2 cal. CFG	op F1
Q1	38	0.754	0.750	0.754	0.750	0.793	0.820	0.061
Q2	38	0.455	0.741	0.455	0.741	0.455	0.784	0.075
Q3	32	0.614	0.751	0.605	0.750	0.607	0.811	–
Q4	21	0.505	0.768	0.505	0.768	0.507	0.870	–

TABLE XV

RQ-L9, primary set P (M2N+V2N, f2, uncalibrated): counted candidates, false ones, and the share of the false ones per class (rules of EXPERIMENT.md).

kind	cand.	false	same ext.	frag. rel.	frag. other	straddle	partial	noise
ff_en	1,070	1,067	0.01	0.99	0.00	0.00	0.00	0.00
mux_bit	344	336	0.00	0.61	0.39	0.00	0.00	0.00
eq_const	1,105	1,096	0.00	0.18	0.82	0.00	0.00	0.00
register	155	116	0.03	0.64	0.15	0.19	0.00	0.00

TABLE XVI

RQ-L9 (H9b), set P: ground-truth instances with at least two cells, the share whose fragment union covers at least 0.8 of them (q), the same with candidates of any kind, and the mean fragment coverage.

kind	GT	matched	q frag.	q any	mean cov.
ff_en	156	3	0.122	0.160	0.618
mux_bit	368	2	0.005	0.136	0.016
eq_const	995	5	0.040	0.112	0.089
register	289	39	0.194	0.228	0.497

not as proof of an innovation. R2-L2 INCONCLUSIVE: Loki is below the model on Q1 and Q2 (AST 0.763 and 0.460 against 0.894 and 0.761) and its output is equal to its input in 150 of 150 runs, but the AST falls from Q1 to Q4, which may partly reflect which references parse, so the data position the two tools (the model readable and unverified, Loki verified and partial) without deciding between them.

E. Why the detectors miss: fragments and words (H9a, H9b)

Hypotheses. H9a: most false candidates are fragments of a word of a related kind. H9b: the fragments of a word cover most of it, so grouping them recovers the word.

Experiment. Uncalibrated, the detectors propose many candidates that almost never match (Table XV); calibrated, they commit almost nothing. Round 3 is compute-only on the round-2 runs and the L0 answer key (no language-model call, no new detector); every figure below is a macro generated from the summary CSVs of experiments/round3. The round-2 counts that motivate the question (for instance ff_en 732 candidates and one correct on the held-out designs) were known when the thresholds were set. Analyses added after the data was seen are labelled exploratory and decide nothing. We added the runs the round-2 sets lacked (micro f3, calibrated and not, and micro f1-k6, uncalibrated; one micro f3 design timed out) and score every set, but the decisions use the primary set P: the micro and VerilogEval f2 runs without calibration.

Result. A false candidate is a counted candidate the one-to-one scorer does not match. We classify each (rules in the protocol): the same extent as a ground-truth instance of its family (a lost tie or a type slip), a fragment of a larger instance of its family, a fragment of an instance of another type, a straddle of several, a partial overlap, or noise. Over the 2,615 false candidates of the four kinds in P, 0.583 (1,525) are fragments of an instance of the matching or a related family. Per kind the related-fragment share is 0.988 for ff_en, 0.638 for register, 0.610 for mux_bit and only 0.175 for eq_const; 3 of four kinds reach 0.5, so H9a is supported by the letter of the rule; not robust. Three qualifications go with it.

First, the noise class is vacuous. The ground-truth instances cover every non-I/O, non-constant cell of the f2 datasets (9,381 of 9,381 in VerilogEval, 9,130 of 9,130 in the micro set), so rule 6 of the protocol could never yield noise, and the noise share 0.000 is structural. The arm of the H9a rule that reads "not supported when the noise share is at least 0.5" was unreachable by construction, and the sentence that no wrong candidate is noise carries no information. As an exploratory replacement we count noise′: a false candidate all of whose cells lie only in plain-gate ground-truth words ($and, $or, $not, $xor, $xnor, the $logic and $reduce gates). Pooled, noise′ is 0.220 (574 candidates); per kind it is 0.522 for eq_const, 0.006 for mux_bit, and 0.000 for ff_en and 0.000 for register (Table XVIII). Half of the false eq_const candidates are thus glue logic, not a piece of a word.

Second, the ff_en fragments are near-tautological. An ff_en candidate has two cells (a flip-flop and its enable LUT), and the ground-truth footprint of a register or $dffe word is the whole word, so a correct per-bit slice of a word counts as a fragment of it; 1,054 of the 1,525 related fragments are ff_en. For ff_en the detector finds correct bit slices and the miss is word-level grouping and extent. Without ff_en (exploratory) the pooled related-fragment share over the other three kinds is 0.304 (1,548 false candidates, 2 of three kinds at or above 0.5), and 0.271 when the largest design is also left out. For mux_bit and register the fragment reading holds at the stated shares;

TABLE XVII
RQ-L9 for every run set (four decisive kinds pooled): runs, candidates, false candidates and the share that are related fragments, other-type fragments, noise.

set	runs	cand.	false	frag. rel.	frag. other	noise
M2N	65	705	686	0.28	0.69	0.00
M2C	65	71	65	0.95	0.05	0.00
V2N	150	1,969	1,929	0.69	0.30	0.00
V2C	not run					
V3N	150	5,005	4,997	0.29	0.71	0.00
V3C	150	2	0	0.00	0.00	0.00
M3N	64	1,406	1,405	0.23	0.77	0.00
M3C	not run					
M1N	68	496	469	0.38	0.57	0.00

TABLE XVIII
Additional, non-deciding analyses, set P: share of false candidates that are related fragments (by candidate, by distinct cell), the noise′ share (cells only in plain-gate ground-truth words), q over all words and over the words touched by a candidate.

kind	frag. rel.	frag. rel. (cells)	noise′	q	q touched
ff_en	0.99	0.98	0.00	0.122	0.123
mux_bit	0.61	0.61	0.01	0.005	0.006
eq_const	0.18	0.18	0.52	0.040	0.042
register	0.64	0.49	0.00	0.194	0.229
pooled	0.58	0.56	0.22	0.065	0.069

for eq_const it is a detector problem: 0.822 of its false candidates are fragments of an instance of another type (mostly single-LUT $and and $or words, registers and fsm instances), so the LUT computes something that looks like a constant compare and is not one. The mux_bit candidates are 0.390 inside an instance of another type ($shiftx words among them).

Third, the result rests on one design. Most ff_en candidates come from ve_Prob153_gshare, where they sit inside a mux tree, and overlapping candidates inflate their count. With that design left out (exploratory, not the rule) 1,863 false candidates remain, 0.415 of them related fragments and 0.563 fragments of another type, so the pooled share falls under the 0.5 of the rule and H9a would be inconclusive; ff_en stays at 0.966 and register drops to 0.333. Counted over the 4,004 distinct (design, cell) pairs covered by false candidates instead of over candidates (exploratory), the pooled related-fragment share is 0.558. We therefore state H9a with both numbers: supported by the letter of the rule at 0.583, not robust (0.415 without ve_Prob153_gshare, 0.304 without ff_en).

H9b asks whether grouping the fragments would recover the word: the share of ground-truth instances whose fragment union covers 0.8 of them is 0.065 pooled (ff_en 0.122, register 0.194, eq_const 0.040, mux_bit 0.005), and no kind reaches 0.5, so H9b is not supported. The denominator holds words no candidate touched; over the 1,698 words touched by at least one candidate (exploratory) the share is 0.069. The shortfall is therefore detector misses plus non-tiling, not one of them: 0.065 of all words, 0.069 of the touched ones. Even with candidates of any kind the cover stays low (register 0.228, mux_bit 0.136). The fragments exist, but they do not tile the words: a merge step over the present detector output would recover few words. The other sets (Table XVII) have fewer candidates when calibrated, and the gate floor f3 proposes far more false eq_const candidates than any f2 set.

TABLE XIX
RQ-L11, ground truth: feedback registers, constant-compare decoders on them, sinks (mux selects and enables) and chains, and the muxes with a select (enabled words) whose select (enable) is a chain sink.

dataset	floor	designs	with fb.	with chain	fb. reg.	fsm flag	decoders	sinks	chains	mux S chain/all	enable chain/all
micro	f2	68	3	1	3	0	11	3	12	3/41	0/3
micro	f3	65	3	1	3	0	11	3	12	3/41	0/3
micro	f1-k6	68	3	1	3	0	11	3	12	3/41	0/3
veval	f2	150	55	12	197	16	41	30	89	15/43	15/152
veval	f3	150	55	12	197	16	41	30	89	15/43	15/152

TABLE XX
RQ-L11, detectors: chain elements proposed by a counted candidate with Jaccard at least 0.5 of any kind (in brackets: of the right scored type), chains with all three elements proposed, and ground-truth FSM instances matched by an fsm candidate.

set	fb. register	decoder	sink	complete chains	fsm
M2N	0/3 (0)	1/11 (0)	1/3 (0)	0/12 (0)	0/0
M2C	0/3 (0)	0/11 (0)	0/3 (0)	0/12 (0)	0/0
V2N	41/197 (26)	15/41 (9)	5/30 (0)	5/89 (0)	6/16
V2C	3/197 (0)	1/41 (0)	2/30 (0)	1/89 (0)	0/16
V3N	13/197 (3)	6/41 (2)	3/30 (0)	4/89 (0)	3/16
V3C	4/197 (0)	2/41 (0)	2/30 (0)	2/89 (0)	1/16
M3N	0/3 (0)	0/11 (0)	0/3 (0)	0/12 (0)	0/0
M3C	0/3 (0)	0/11 (0)	0/3 (0)	0/12 (0)	0/0
M1N	0/3 (0)	0/11 (0)	0/3 (0)	0/12 (0)	0/0

RQ-L11: the FSM chain. Measurement only. In the f2 ground truth of VerilogEval (150 designs) 197 registers feed back to themselves, of which 16 the front end flags as FSM state; 41 constant-compare decoders read such a register, and the chains (register, decoder, select or enable) occur in 12 designs. Of the muxes with a select, 15 of 43 (VerilogEval) and 3 of 41 (micro) take it from such a decoder, 0.214 together, so the FSM, decode, select chain is real but covers a minority of the control muxes of these small designs. The detectors see little of it (Table XX): in V2N they propose 41/197 feedback registers, 15/41 decoders, 5/30 sinks, all three elements of 5/89 chains, and 6/16 FSM instances as an fsm candidate. The chains are triples (register, decoder, sink) and reuse their elements: the 89 triples hold only 30 distinct sinks and 41 distinct decoders. Counted on distinct elements, the detectors propose 5/30 of the sinks and 15/41 of the decoders, against 5/89 complete triples.

Implication for a word-first detector. The classification cannot say that the wrong candidates are not noise, because the ground truth covers every cell and the noise class was unreachable; the exploratory noise′ share is 0.220 pooled and 0.522 for eq_const. The fragment share depends on the kind and, for the pooled figure, on one design and on the ff_en slices: ff_en, mux_bit and (less so) register fail as pieces of larger constructs, while eq_const

fails as a detector problem (the local pattern is too weak, and a word-level view, the select and enable fan-out across a word, must decide whether a compare is a decoder). Because the fragments do not tile the words, grouping the present candidates is not enough, and a detector should construct the word first and score it as a unit. Concretely, we propose to find (i) register words by shared clock, enable and reset across flip-flops; (ii) datapath words by per-bit cones that are NPN-isomorphic, seeded from the ports but not limited to them (the port words reach registers and nothing past the first layer); (iii) control nets by select and enable fan-out across a word; and (iv) FSMs by a feedback strongly connected component plus the decoders on it. Related work attacks the word problem along the same lines: WordRev [11] derives word-level structures from the gate-level netlist of a circuit and lets the user specify sequences of word-level operations whose gates it then extracts, and DANA [7] recovers high-level registers by analysing the flow of data between individual flip-flops. We ran DANA (through HAL [5]) on the round-5 sets (Section V-F); WordRev was not run and no comparison with it is claimed. The next round builds this detector (L8, the language model given the ReverseProvenance record, stays deferred).

Verdicts. H9a POSSIBLY GOOD: by the letter of the rule about half of the false candidates are fragments of a related kind (0.583 pooled), but the result fails leaving out one design (0.415) and the near-tautological ff_en slices (0.304), so we state it only with that fragility. H9b DISMISSED: a word's fragments cover it in 0.065 of the words, so the fragments exist but do not tile the words; this is the motive for the word-first detector of the next subsection. The FSM chain is a measurement and has no verdict.

F. The word-first detector and DANA (H16a–H16f, RQ-H1)

Hypotheses. The word-first detector, built from the two lessons of the fragment study, makes six claims: H16a, word tiling beats the earlier detectors on held-out real designs; H16b, grouping by clock, enable and reset recovers the enabled-register words; H16c, a word-level eq_const beats the per-bit one; H16d, the FSM-decode-select chain is completed more often; H16e, words grown from registers reach the datapath beyond a size-matched null; H16f, repetition is evidence on big designs. RQ-H1: the detector recovers register words better than, equal to, or worse than DANA.

The fragment study says what to build: words from structure. Round 5 builds it (can words, control nets and FSMs be found first, and the rest built around them?). Two opt-in detectors, wordfirst and wordgroup (loki/re/detectors/wordfirst.py, docs/user/wordfirst.md), were added to loki re; the default detector list and every earlier calibration are unchanged. wordfirst groups flip-flops by clock, enable and reset into register words (with ff_en and ff_srst words labelled by the control net), groups 2:1 mux cells by their shared select, takes the carry chains of the arith detector so that the carry net joins the bits, decides eq_const at word level (a single-minterm cone whose support lies in one register word or the primary inputs), proposes an FSM for a register word coupled through its own D cones, and grows words along isomorphic bit slices (readers and drivers) across registers, mux words and adders into bus words. wordgroup runs last and links the per-bit hypotheses to the words that contain them. The outputs are ordinary candidates with claims, so the prover, the commit step, the scorers and the Verilog writer work unchanged. Every threshold is an agent default (no fitted parameter): groups above 32 flip-flops are split by the Q-reader and D-driver signature, eq_const supports are at most 16, growth needs 0.75 of the bits and two levels.

TABLE XXI
Round 5: word tiling q (share of GT words whose fragments cover 0.8) at f2.

Set	R2N	R2C	W	WB	null W	null WB
MI	0.212	0.019	0.223	0.237	0.042	0.048
VT	0.022	0.004	0.017	0.023	0.017	0.203
CI	0.390	0.328	0.312	0.397	0.206	0.294
CO	0.259	0.182	0.180	0.266	0.089	0.155

The evaluation sets are the micros (MI), the 33 held-out VerilogEval designs (VT), the corpus iid and ood test splits (CI, CO) and the 13 big designs that built at f2 (BIG). The arms are the round-2 detectors uncalibrated and calibrated (R2N, R2C), wordfirst alone (W) and wordfirst with the per-bit detectors and wordgroup (WB); all runs are uncalibrated.

Experiment and result. H16a: tiling. On VT the pooled tiling of WB is 0.023 against 0.022 for the better round-2 arm (difference 0.001, bootstrap interval 0 to 0.062, 1,388 instances over 17 designs, Wilcoxon p 1). Word grouping does not tile the held-out real words; on the micros and the synthetic splits the tiling of WB and of the round-2 arm are close (Table XXI). The null columns replace each counted candidate by random cells of the same size. On VT the null of WB (0.203) is above WB itself and above every arm (the largest arm value is 0.023), so the null clause of the decision rule (q(WB) minus the null at least 0.10) cannot hold on VT. With the VT baseline the difference clause needs q(WB) of 0.172 and the null clause 0.303, so a supported verdict needed q(WB) of about 0.303. The status does not depend on the null: the difference is below 0.05. The null depends on the size mix of the candidates: WB has 34 counted candidates of at least ten cells on VT against 20 for R2N, the largest has 1,016 cells (R2N: 64), and the null of all of WB's counted candidates is 0.203 against 0.203 for R2N, while the ff_en candidates alone give 0.192 for WB and 0.004 for R2N (exploratory).

TABLE XXII
Round 5, floor f2: precision/recall/F1 per kind and set for the round-2 detectors (R2N, R2C) and the word-first arms (W, WB). Pooled over the designs each arm has; GT is the first arm's count.

Set	Kind	GT	R2N	R2C	W	WB
CI	adder	49	1.00/0.14/0.25	1.00/0.14/0.25	1.00/0.14/0.25	1.00/0.14/0.25
CI	comparator	167	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00
CI	counter	10	0.62/0.50/0.56	1.00/0.60/0.75	0.00/0.00/0.00	0.62/0.50/0.56
CI	eq_const	161	0.02/0.02/0.02	1.00/0.02/0.04	0.04/0.02/0.02	0.02/0.02/0.02
CI	ff_en	78	0.00/0.00/0.00	0.00/0.00/0.00	0.20/0.18/0.19	0.02/0.18/0.03
CI	ff_srst	70	0.00/0.00/0.00	0.00/0.00/0.00	0.31/0.39/0.35	0.04/0.37/0.07
CI	fsm	20	0.89/0.80/0.84	1.00/1.00/1.00	0.09/0.05/0.06	0.58/0.75/0.65
CI	mux	106	0.73/0.08/0.14	1.00/0.07/0.12	0.67/0.06/0.10	0.40/0.08/0.13
CI	mux_tree	19	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00
CI	register	163	0.57/0.37/0.45	0.59/0.26/0.36	0.27/0.30/0.29	0.36/0.40/0.38
CI	subtractor	32	0.50/0.09/0.16	1.00/0.09/0.17	1.00/0.09/0.17	0.50/0.09/0.16
CO	adder	90	0.22/0.02/0.04	0.22/0.02/0.04	0.30/0.03/0.06	0.22/0.02/0.04
CO	comparator	502	0.03/0.01/0.01	1.00/0.01/0.01	0.00/0.00/0.00	0.03/0.01/0.01
CO	counter	13	1.00/0.08/0.14	1.00/0.08/0.14	0.00/0.00/0.00	1.00/0.08/0.14
CO	eq_const	491	0.01/0.01/0.01	0.33/0.01/0.02	0.01/0.01/0.01	0.01/0.01/0.01
CO	ff_en	171	0.00/0.00/0.00	0.00/0.00/0.00	0.24/0.22/0.23	0.02/0.22/0.03
CO	ff_srst	87	0.00/0.00/0.00	0.00/0.00/0.00	0.20/0.51/0.29	0.02/0.48/0.04
CO	fsm	40	0.88/0.53/0.66	1.00/0.97/0.99	0.06/0.03/0.04	0.54/0.53/0.53
CO	mux	213	0.32/0.08/0.13	0.94/0.08/0.15	0.46/0.03/0.05	0.49/0.08/0.14
CO	mux_tree	32	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00
CO	register	304	0.52/0.37/0.43	0.57/0.28/0.38	0.29/0.34/0.31	0.36/0.42/0.39
CO	subtractor	38	0.17/0.03/0.05	1.00/0.03/0.05	1.00/0.03/0.05	0.20/0.03/0.05
MI	adder	62	1.00/0.13/0.23	1.00/0.11/0.20	0.75/0.13/0.23	1.00/0.13/0.24
MI	comparator	116	0.01/0.01/0.01	0.00/0.00/0.00	0.00/0.00/0.00	0.01/0.01/0.01
MI	counter	2	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00
MI	eq_const	102	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00
MI	ff_en	4	0.04/0.50/0.07	0.00/0.00/0.00	0.14/0.25/0.18	0.05/0.75/0.09
MI	ff_srst	9	0.00/0.00/0.00	0.00/0.00/0.00	0.11/0.33/0.16	0.02/0.33/0.03
MI	fsm	0	–	–	0.00/0.00/0.00	–
MI	mux	49	0.14/0.02/0.04	0.14/0.02/0.04	0.00/0.00/0.00	0.12/0.02/0.04
MI	mux_tree	3	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00
MI	register	94	0.28/0.12/0.16	0.00/0.00/0.00	0.18/0.27/0.22	0.22/0.16/0.19
MI	subtractor	27	0.50/0.07/0.13	1.00/0.07/0.14	1.00/0.04/0.07	0.50/0.07/0.13
VT	adder	7	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00
VT	comparator	547	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00
VT	counter	6	0.67/0.33/0.44	1.00/0.33/0.50	0.00/0.00/0.00	0.67/0.33/0.44
VT	eq_const	541	0.01/0.00/0.00	0.00/0.00/0.00	0.03/0.00/0.00	0.01/0.00/0.00
VT	ff_en	130	0.00/0.01/0.00	0.00/0.00/0.00	0.01/0.02/0.02	0.00/0.02/0.00
VT	ff_srst	6	0.00/0.00/0.00	0.00/0.00/0.00	0.03/0.83/0.06	0.01/0.83/0.02
VT	fsm	4	0.40/0.50/0.44	0.00/0.00/0.00	0.00/0.00/0.00	0.33/0.25/0.29
VT	mux	22	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00
VT	mux_tree	2	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00
VT	register	146	0.11/0.05/0.07	0.00/0.00/0.00	0.06/0.06/0.06	0.05/0.06/0.06
VT	subtractor	2	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00

The tiling measure cannot separate structure from chance when a few candidates are large.

H16b: ff_en words. Pooled over all four sets the recall of ff_en rises from 0.008 (best round-2 arm) to 0.146 for WB (precision 0.001 to 0.014); the gain in recall (0.138) and in F1 (0.025) is below the margins of the rule (0.20 and 0.10) although the paired test over 113 designs is significant, so the status is inconclusive. Grouping by shared control matches some enabled words that per-bit slices never matched, at a precision of 0.014 (F1 0.026); the recall stays at 0.146. Why it stops there was not measured. Three reasons are plausible and untested: the enable is often a function of several nets, the cone depth limit of the ff_pattern analysis cuts it, and the ground-truth footprints of such words are whole arrays that share cells.

H16c to H16f. The word-level eq_const changes nothing measurable (F1 0.001 for WB against 0.001 for the baseline; it is a subset of the per-bit rule). The chain test is inconclusive (WB completes 3 of 22 chains, the round-2 arm 1) and the FSM rule that was evaluated over-generates. Growth from register words reaches the datapath at 0.788 (WB) but a size-matched random null reaches 0.715, so the level alone is not evidence. Repetition on the big designs lowers F1 (0.013 to 0.010), and H16f is degenerate: no outcome but not supported was reachable. Their details are in the repository (experiments/round5) and in the research log.

Limits of the detector. It reuses the ff_pattern cone analysis and the arith chain finder, so its adders are those of the baseline. At f3 every arm has zero recall on muxes and comparators. The rules were developed on 150 synthetic train designs and did not transfer to the real held-out set.

Against DANA. RQ-H1 asks whether an established word finder does better on the same netlists. The decision rule, the scorer and the bootstrap interval are written in the repository's HAL comparison plan. The tool is HAL v4.5.0 (commit e7fea566b51e) with its dataflow plugin DANA [5], [7], built without the graphical interface. Both tools read the same netlist.blif of a floor; a converter turns it into structural Verilog of the cells of one of two gate libraries that we wrote for HAL (a LUT library for f2, the cmos6 library for f3), with the instance name equal to the Odin III node id so that a DANA group is a set of answer-key nodes. The HAL v4.5.0 command line –dataflow does not run (its handler sets no gate types), so DANA was driven through the plugin's Python API with its documented default configuration; only min_group_size was set. The library default of min_group_size (8) is the headline arm; 2 is a sensitivity run. Loki's side is the round-5 W arm, not rerun. Ground truth are the L0 storage words with WIDTH ≥ 2 whose footprint, cut to flip-flops, holds at least two flip-flops, scored by one code path for both tools (Jaccard-0.5 one-to-one matching, exact match, purity, completeness).

Table XXIV gives the pooled result on MI and VT. On f2 the pooled F_J of DANA is 0.261 against 0.169 for Loki's W arm over 100 designs (41 of them with a ground-truth word); the difference $D = F_J(\text{Loki}) - F_J(\text{DANA})$ is -0.092, with a 95% cluster-bootstrap interval of [-0.317, 0.043]. The decision rule returns inconclusive: the point estimate favours DANA, but the interval is wide (the micro designs come in template families), the paired Wilcoxon test over the designs with a ground-truth word does not reach $p < 0.05$ (0.172 for DANA greater), and the interval is not inside the equivalence band either, so neither a lead nor a tie is shown. The comparison scorer (duplicate predictions counted once, Loki's cells and the ground-truth footprints cut to flip-flops) moves Loki's pooled f2 F_J from 0.129 to 0.169. The decision is therefore shown under both scorers (Table XXIII; DANA's counts are the same in both rows): D is -0.092 with the comparison scorer (95% interval [-0.317, 0.043]) and -0.132 with the round-5 scorer ([-0.382, -0.044]); the rule applied to each alone gives inconclusive and inconclusive, and neither scorer is the headline. DANA's size-2 run gives a pooled F_J of 0.306; it is a sensitivity run, never the headline. Its outcome is also not a decided one, but with the round-5 scorer alone the rule gives HAL ahead for it (comparison scorer alone:

inconclusive).

The two tools differ in kind. DANA proposes fewer groups (51 against 268 for Loki on f2); a larger share of them matches a word (0.745 against 0.160), but few words are recovered by either tool (recall 0.158 for DANA, 0.179 for Loki). DANA's groups cover most flip-flops of the ground-truth words (0.985 against 0.843) and are more complete (0.550 against 0.063), with a lower purity (0.853 against 0.907). Its groups are larger (median 10.0, mean 37.3 flip-flops, against 2.0 and 6.0 for Loki on f2), and 0.25 of them (13) merge words, that is, hold flip-flops of a ground-truth word and of another one, against 0.07 (18) for Loki. Loki's groups are purer but split words and leave flip-flops out. On f3 DANA's rows equal its f2 rows: it reads the flip-flop graph and control nets only, and the f2 and f3 netlists of a design have the same flip-flops and the same flip-flop connectivity; DANA's partitions are equal on 98 of 98 designs (under the original flip-flop names) and no netlist holds a DFFSR cell (0 in 199). Loki's f3 numbers differ from f2 (Table XXIV); at f3 Loki's precision is higher and its recall lower. This shift comes mostly from the scorer's flip-flop cuts (Loki's cells and the ground-truth footprints), which raise the match count only together, and not from the detector: with the round-5 scorer Loki's pooled f3 F_J is 0.065 (precision 0.149, recall 0.042), against 0.226 with the comparison scorer (docs/results-hal.md).

Limits. The 13 large designs were not scored: 7 contain a RAM hard block that neither gate library can express, which is above the 20% drop limit; DANA ran on the 6 that convert (at most 46 s each) and the group counts are in docs/results-hal.md. No evaluation netlist contains a flip-flop with set and reset pins, so DANA's control-net evidence is the clock only on both floors; a netlist with enable and reset flip-flops is the case DANA was designed for, and we did not test it. The converter was checked against the BLIF by simulation (micro, VerilogEval and the convertible large designs); the 10545 LUT truth tables as HAL parses them show 0 mismatches with the BLIF, and the BLIF and HAL LUT counts differ on 0 of 100 designs; DANA returned the same groups on a second run (5 of 5 designs, and 5 of 5 with both runs uncached) and with permuted instance names and shuffled instance order (6 of 6 designs); these two checks cover only small VT designs (the 5 smallest designs with a ground-truth word, and 6), no MI design; HAL was not patched and DANA was not tuned. The ground truth is the Odin III answer key, so a group that matches no word is wrong only in that sense. The result is limited to register words on these sets: under the decision rule the word-first rules are not shown to be ahead of, or equal to, a published dataflow method, and neither tool recovers most of the words.

Verdicts. H16a DISMISSED (tiling 0.023 against 0.022 for the baseline; the null is 0.203). H16b INCONCLUSIVE (recall rises from 0.008 to 0.146 but F1 only by 0.025, below the margins, at a precision of 0.014). H16c and H16f DISMISSED; H16d and H16e INCONCLUSIVE. RQ-H1 INCONCLUSIVE: pooled F_J 0.169 for Loki against 0.261 for DANA, difference -0.092 with 95% interval $[-0.317, 0.043]$ and Wilcoxon p(Loki greater) $= 0.8284$; neither a lead nor a tie is shown, and neither tool recovers most of the words. We therefore make no claim that the word-first detector is an innovation that works.

TABLE XXIII
The decision on E_real at f2 under the two scorers for Loki (HAL counts identical in both rows). $D = F_J$(Loki) $- F_J$(DANA); p_L, p_H: one-sided Wilcoxon p, Loki greater and DANA greater. The last column applies the rule to that row alone; the decision needs Loki ahead under both.

Scorer	D	95% CI	90% CI	p_L	p_H	rule alone
Comparison scorer	-0.092	[-0.317, 0.043]	[-0.266, 0.009]	0.828	0.172	inconclusive
Round-5 scorer	-0.132	[-0.382, -0.044]	[-0.337, -0.052]	0.999	0.001	inconclusive

G. Where synthesis loses constructs: the loss audit (L15a, L15c, L15d)

Hypotheses. L15a: each construct kind is first lost at one dominant Odin III pass. L15c: the D14 labels explain the merged losses. L15d: a pass can fool a detector (its recall falls across the pass).

Experiment and result. The word-first detector finds register and enable words and does not find FSM decoders, compares and mux boundaries; we guessed that they are absorbed into LUTs at f2. The audit asks where and why: for each HDL construct, at which Odin III step is it first lost as a separable unit, joined with what, and why? The measure is a count of constructs lost per kind and pass, the share that the D14 provenance explains, and the recall of the round-5 detectors before and after a pass. It is not area, speed or power. The tools are Odin III [12] at c82609e6, which writes the version 2 name map (–prov-map: records, join labels and the construct table), and ABC [13] inside its abc pass; no VPR run is involved.

Protocol. Development used hand-written fixtures and the corpus train split. The f2 flow of round 5 is run pass by pass in one Python session (proc, fsm, opt, flatten, opt -full, wreduce, lower, abc) and a netlist plus map is written after each pass. A unit is the statement-level anchor of a record's construct leaves (V1.2.0 Section 3.3), reduced by dropping ancestors; a unit is separable at a snapshot when some logic object has exactly that unit set, and lost at the first later snapshot where none has. A loss is explained when a labelled same-pass join record names the unit; its reason class is read from the pass and that evidence. The map holds one record per object (checked on all 214,157 objects of the 1,768 maps), so reading the first record of an object loses nothing.

Two nulls. The random-set null draws, for every real object of a pass, a random unit set of the same size from the units alive at the pass. It matches the object count and the size distribution of the pass; it does not match

TABLE XXIV

Loki word-first detector (W) against HAL's DANA on MI and VT (E_real), word recovery against the L0 ground truth; one scorer for both tools. P_J, R_J, F_J: Jaccard-0.5 matching; purity and completeness over flip-flops. The round-5 scorer row counts Loki as round 5 did (the scorer used for Loki in round 5). Loki WB has a run on fewer designs than the other arms (f3: 96 of 98 designs; listed in docs/results-hal.md), so its rows pool a different design set.

	f2 (LUT6)					f3 (cmos6)				
Arm	P_J	R_J	F_J	pur.	compl.	P_J	R_J	F_J	pur.	compl.
Loki W	0.160	0.179	0.169	0.907	0.063	0.557	0.142	0.226	0.952	0.850
Loki W, round-5 scorer	0.119	0.142	0.129	-	-	0.149	0.042	0.065	-	-
Loki WB	0.141	0.133	0.137	0.983	0.129	0.320	0.071	0.116	0.956	0.874
DANA, size 8 (headline)	0.745	0.158	0.261	0.853	0.550	0.745	0.158	0.261	0.853	0.550
DANA, size 2	0.613	0.204	0.306	0.865	0.549	0.613	0.204	0.306	0.865	0.549

TABLE XXV

Round 6: units per set at the stmt level and what became of them.

Set	Designs	Units	Kept	Lost	Born joined	Capped
ALL	221	1436	474	580	382	21
CI	120	765	179	304	282	20
MI	68	546	253	193	100	1
VT	33	125	42	83	0	0

TABLE XXVI

Round 6: first losses (the total under the random-set null) and, apart from them, the subsumed ones (the unit survives only as the enclosing statement of a finer unit that is still alive) and the remaining merged-plus-removed losses (stmt level, MI+VT+CI).

Pass	First losses	Subsumed	Share	Merged + removed
S3_opt	320	127	0.40	193
S4_flatten	68	0	0.00	68
S5_opt_full	26	1	0.04	25
S8_abc	166	33	0.20	133
all	580	161	0.28	419

the chains, because it forgets which units the objects of the previous snapshot already held together. A pass that changes nothing still draws fresh random sets and "loses" most units, so the random-set null overstates chance losses. The persistence null starts every draw from the real objects of the previous snapshot and applies random merges until the object count of the real snapshot is reached (50 draws; mean and a 95% interval of the per-draw totals; a pass that changes nothing gives zero; a pass that only removes dead objects is read as merges, so at opt it is an upper bound of chance merging). The rule thresholds are the same for both nulls. The sets are the 68 micros (MI), the 33 held-out VerilogEval designs (VT) and the 120 corpus iid designs (CI), each with all designs ok; the 13 big designs and the ood split were not run. The snapshot writers do not change the netlist: the final BLIF of all 68 micros (hash-style names, MI only) is byte-identical to a run without them.

Where units are lost. Of 1436 units, 580 are lost, 474 are never lost and 382 are joined from the first snapshot onwards (Table XXV). The losses fall on four passes only: 320 at opt, 68 at flatten, 26 at the second opt and 166 at abc. None is lost at fsm, wreduce or lower: lower splits word cells into gates, which makes units finer, not blended. The 580 is the total. It includes 161 subsumed losses (a statement that stays alive only as the enclosing statement of a finer unit; 0.28 of the total, 127 of the 320 at opt and 33 of the 166 at abc); without them the merged-plus-removed losses are 419 (Table XXVI). The 68 at flatten are all INSTANCE units (the instance cell of a dissolved module, class removed) and say nothing about logic: the logic kind is dominant at flatten because of them, not because dead logic is removed.

By hardware kind (Table XXVII), under the random-set null, registers (dominant) and compares (dominant) are lost at opt (84 of 89 and 18 of 19, against random-

TABLE XXVII

Round 6: first losses of a unit per hardware kind and pass, as real/random-set null/persistence null (expected counts, stmt level, MI+VT+CI). The first column is the units at risk at opt. The random-set null is persistence-free and overstates chance losses; the persistence null (mean of 50 draws) starts from the real objects of the previous snapshot. Both nulls and the real count use the same units at risk (separable at the previous snapshot, not lost earlier). The logic loss at flatten is the 68 INSTANCE units (removed), not dead logic. The loss measure cannot see FSM re-encoding (hardware kind fsm is read at S1, before the fsm pass), so the fsm row is empty and FSM units are counted under register, mux or logic.

Kind	n	fsm	opt	flatten	opt -full	wreduce	lower	abc
fsm	0	–	–	–	–	–	–	–
register	218	0/59/0	84/94/55	0/35/0	4/35/4	0/31/0	0/7/0	1/13/9
arith	172	0/37/0	17/51/22	0/36/0	7/36/0	0/35/0	0/0/0	31/22/20
mux	359	0/108/0	192/223/216	0/57/0	14/61/5	0/45/0	0/10/0	79/36/52
compare	33	0/6/0	18/14/16	0/3/0	0/1/0	0/2/0	0/1/0	1/1/3
logic	272	0/71/0	9/81/14	68/58/4	1/38/5	0/39/0	0/13/0	54/46/50

TABLE XXVIII

Round 6: real first losses / random-set null / persistence-null mean [2.5th–97.5th percentile of the 50 per-draw totals], stmt level, MI+VT+CI. The interval covers the draws within the fixed designs, not variation across designs. A pass that removes objects lowers the object count and the persistence null reads every removal as a merge, so at opt it is an upper bound of chance merging.

Kind	opt	flatten	opt -full	abc
fsm	–	–	–	–
register	84 / 94 / 55 [48–64]	0 / 35 / 0 [0–0]	4 / 35 / 4 [1–7]	1 / 13 / 9 [6–12]
arith	17 / 51 / 22 [15–29]	0 / 36 / 0 [0–0]	7 / 36 / 0 [0–2]	31 / 22 / 20 [17–23]
mux	192 / 223 / 216 [202–226]	0 / 57 / 0 [0–0]	14 / 61 / 5 [2–10]	79 / 36 / 52 [47–58]
compare	18 / 14 / 16 [11–20]	0 / 3 / 0 [0–0]	0 / 1 / 0 [0–1]	1 / 1 / 3 [2–3]
logic	9 / 81 / 14 [9–19]	68 / 58 / 4 [4–4]	1 / 38 / 5 [3–7]	54 / 46 / 50 [45–56]

TABLE XXIX
Round 6: first losses per reason class and pass (stmt level, MI+VT+CI). Classes with no loss are kept.

Reason	fsm	opt	flatten	opt -full	wreduce	lower	abc
removed	0	12	68	7	0	0	10
subsumed	0	127	0	1	0	0	33
fsm-recode	0	0	0	0	0	0	0
instance-join	0	0	0	0	0	0	2
opt-merge	0	77	0	17	0	0	0
width-reduce	0	0	0	0	0	0	0
lower-share	0	0	0	0	0	0	0
lut-cone	0	0	0	0	0	0	99
unlabelled-merge	0	104	0	1	0	0	22

TABLE XXX
Round 6: merged first losses, those a labelled same-pass join record names (explained), the unlabelled rest and how many of those are inherited blends (exploratory; by descent through the parent records, then by the earlier subset check). stmt level, MI+VT+CI.

Pass	Merged	Explained	Share	Unlabelled	Inherited (descent)	Inherited (subset)
S3_opt	181	77	0.425	104	104	104
S5_opt_full	18	17	0.944	1	1	1
S8_abc	123	101	0.821	22	22	22
all	322	195	0.606	127	127	127

set null shares of 0.34 and 0.49), muxes at opt (192 of 285, null share 0.41) and arithmetic at abc (31 of 55, null share 0.10). By the count rule against the random-set null, abc loses muxes more than chance (79 against 36) and arithmetic at chance, whereas arithmetic at opt (17 against 51) and muxes at the second opt (14 against 61) are below chance. These readings do not survive the persistence null (exploratory, Tables XXVIII and XXXI): opt is also where a chance merging of the objects puts most of its losses, so the pass shares of register, compare, mux and arithmetic are dominant-null, the growth artefact of rule 1 (null shares 0.80, 0.84, 0.79 and 0.46). Only the flatten loss of logic (the instance cells) stays dominant. By the count rule, arithmetic at opt is at chance (17 against a mean of 22, interval 15 to 29), not below; the "below chance" reading for it and for muxes at the second opt comes from the missing persistence (for muxes at the second opt, 14 against 5 with interval 2 to 10, the real count is above the interval). Registers at opt (84 against 55, interval 48 to 64) and arithmetic and muxes at abc (31 against 20, interval 17 to 23; 79 against 52, interval 47 to 58) are above the interval but within a factor two of the mean, so the count rule reads them as at chance. The supported statement is therefore descriptive: a compare statement is no longer a separable unit after the first opt, before any LUT is formed (18 of 19), but that the first opt is the pass that loses them is what a chance merging of the same number of objects also gives, so the data do not show that compares are lost there more than other kinds. The round-5 guess that the LUT cone is where compares go is not supported either: one compare unit is lost at abc. The branch level gives the same picture (548 losses against 580; l15_kind_pass.csv). The persistence null matches the size profile of the real snapshot only approximately:

TABLE XXXI
Round 6, rule 1: the pass that loses most of each hardware kind, its share, and the outcome under the random-set null (persistence-free) and, beside it, under the persistence null with the same thresholds (stmt, MI+VT+CI). Where they differ, neither is confirmatory.

Kind	Pass	Losses	Share/random-set null	Random-set	Share/persist.	Persist. (expl.)
arith	S8_abc	55	0.564/0.101	dominant	0.564/0.462	dominant-null
compare	S3_opt	19	0.947/0.489	dominant	0.947/0.844	dominant-null
logic	S4_flatten	132	0.515/0.168	dominant	0.515/0.055	dominant
mux	S3_opt	285	0.674/0.412	dominant	0.674/0.790	dominant-null
register	S3_opt	89	0.944/0.343	dominant	0.944/0.801	dominant-null

TABLE XXXII
Round 6: recall per kind of the three arms on the gate netlist (S7) and the LUT netlist (S8), MI+VT+CI, on the paired designs (those with a run at both snapshots). GT is the WB ground-truth count at S7/S8; the two differ because the footprints are rebuilt from each snapshot's map (the S7 and S8 ground truths are not the same instances), and the gate and LUT footprints are not like for like.

Kind	GT S7/S8	B S7	B S8	W S7	W S8	WB S7	WB S8
ff_en	212/212	0.00	0.01	0.16	0.11	0.16	0.12
ff_srst	85/85	0.00	0.00	0.53	0.49	0.53	0.47
register	397/397	0.13	0.20	0.22	0.23	0.22	0.24
adder	110/100	0.00	0.15	0.00	0.14	0.00	0.16
subtractor	63/60	0.00	0.08	0.00	0.05	0.00	0.07
comparator	829/829	0.02	0.00	0.00	0.00	0.02	0.00
eq_const	803/803	0.00	0.01	0.00	0.00	0.00	0.01
mux	174/166	0.53	0.06	0.36	0.04	0.55	0.07
mux_tree	24/24	0.04	0.00	0.00	0.00	0.04	0.00
counter	18/18	0.00	0.39	0.00	0.00	0.00	0.39
fsm	24/24	0.12	0.75	0.04	0.04	0.17	0.67

l15_null_size_mad.csv gives the mean absolute deviation of the sorted object sizes, over the designs where merges are drawn (0.90 at opt, 0.88 at abc, 0.32 at the second opt).

FSMs. The loss measure cannot see an FSM re-encoding. The hardware kind of a unit is read at the first snapshot, after proc and before the fsm pass, where no object has the type $fsm; FSM units are therefore counted under register, mux or logic, and the fsm row of Table XXVII is empty. "No unit stops being separable at fsm" is a statement about that pass only; it does not say that FSMs are not lost, and the FSM-recode reason class has no observation.

Why. Table XXIX gives the reason classes. At opt, 77 losses are opt merges that a labelled join record names, 12 are removals, 127 are subsumptions and 104 are merges with no label; at abc, 99 are LUT-cone merges that a record names, 33 subsumptions, 10 removals, 2 instance joins and 22 unlabelled merges. Of the 322 merged losses, 195 (0.606) are explained by a same-pass join record, which the rule reads as partly explained (Table XXX). The share differs by set and by pass: MI 0.978 (88 of 90), CI 0.462 (79 of 171) and VT 0.459 (28 of 61); opt 0.425 (77 of 181, mostly unexplained by the rule), second opt 17 of 18 and abc 0.821 (101 of 123, partly explained). The VT share at opt is 4 of 36; the VT designs are read through the slang adapter, and a gap in the labels that the adapter's records carry would give such a low share (not tested here). The pooled status is as the rule says.

TABLE XXXIII
Round 6: eight lost micro constructs, 'construct at file:line lost at pass P because merged with Q' (first rows of l15_lost_list.csv with one labelled partner).

Design	Construct	Kind	Pass	Joined with
bm_expr_all_mod	bm_expr_all_mod.v:124.2-124.29	NONBLOCKING_ASSIGN	opt	NONBLOCKING_ASSIGN 146.2-146.122
bm_expr_all_mod	bm_expr_all_mod.v:133.2-133.39	NONBLOCKING_ASSIGN	opt	NONBLOCKING_ASSIGN 147.2-147.126
bm_lpm_all	bm_lpm_all.v:44.8-44.20	NET_ASSIGN	opt	NET_ASSIGN 51.8-51.43
bm_match1_str_arch	bm_match1_str_arch.v:49.8-49.26	NET_ASSIGN	opt	NONBLOCKING_ASSIGN 55.2-55.22
bm_match1_str_arch	bm_match1_str_arch.v:50.8-50.26	NET_ASSIGN	opt	NONBLOCKING_ASSIGN 56.2-56.22
bm_match1_str_arch	bm_match1_str_arch.v:51.8-51.26	NET_ASSIGN	opt	NONBLOCKING_ASSIGN 57.2-57.22
bm_match1_str_arch	bm_match1_str_arch.v:55.2-55.22	NONBLOCKING_ASSIGN	opt	NET_ASSIGN 49.8-49.26
bm_match1_str_arch	bm_match1_str_arch.v:56.2-56.22	NONBLOCKING_ASSIGN	opt	NET_ASSIGN 50.8-50.26

All 127 unlabelled merges fit an inherited blend reading (exploratory, by descent through the parent records of the map): for each of them every object that holds the unit has a parent record of the pass that already blended the unit with at least one other unit, so no new join was made and none is labelled (V1.2.0 Section 3.3). The earlier subset check, which looked for any impure object, gives the same 127 of 127. The descent test shows that the unit was already in a blend in a parent; it does not show that every other unit of the new object came from the same parent, so it does not support a claim that the D14 labels name every new blend. "Style collapse" (a case, an if and a ternary of one function) leaves no record and is reported as not observable. Table XXXIII shows eight lost micro constructs with their partner; l15_lost_list.csv holds all of them.

Detectors. The word-level snapshots are not netlists a detector reads, so recall is compared at the two netlist snapshots, the gates after lower (S7) and the LUTs after abc (S8), for the default list (B), wordfirst (W) and the combined list (WB). The ground truth is the round-5 derivation on the word cells of S6 with the span-subset footprint rule of L0 applied to construct keys. The gate footprints (many gates share a construct set) and the LUT footprints are not like for like, and the S7 and S8 ground truths differ in size for some kinds (Table XXXII). For the mux word the ground truth is 174 instances at S7 and 166 at S8, because the footprints are rebuilt from each snapshot's name map, so the two recalls are not fractions of the same set of instances; recall at the two snapshots is compared through rule 4 only. The table uses the paired designs (a run at both snapshots; three designs drop out: one mux design (ve_Prob021_mux256to1v, killed at S7 by the 6 GB slice cap) in all arms and two micro WB runs at S7 that timed out). Under rule 4 the mux word is the only kind that is found at S7 and lost at S8 (fooled by abc in all three arms; WB: recall 0.55 to 0.07 on 174 instances at S7 and 166 at S8). The related kind mux_bit falls from 0.201 to 0.012 (35 of 174 to 2 of 166) in arms B and WB, and rule 4 marks it not fooled: the rule calls a detector fooled only when recall falls by 0.20 or more, and the fall is 0.189, just under that line (arm W finds none at either snapshot). The label not fooled here is therefore a statement about the cut-off, not about robustness. The mux word statement stands for the mux word only: comparator, eq_const, counter, adder, subtractor and mux_tree are at 0.00 to 0.04 at S7, a floor at which a pass cannot fool a detector, so "not fooled" there means "nothing to lose", not "robust". FSM (WB: 0.17 to 0.67 on 24 instances) and counter (0.00 to 0.39 on 18) are found more often at S8 than S7, and nothing else moves by 0.20. The counts are small (mux 174, FSM 24, counter 18 instances).

What this suggests. The map already names, for every LUT, the set of statements it blends, so a detector could split a LUT region by that set instead of guessing; the mux fooling at abc and the compare loss at opt say where such a split is worth the most. This is a suggestion, not a result: no detector was changed in this round. Not done: the big designs, the ood split, a VPR flow, the instance axis across several instances of one module (units are keyed by construct, so two instances of one module are one unit), and the word level, which the map does not observe.

Verdicts. L15d POSSIBLY GOOD, for one kind: the mux word is found at the gate snapshot and lost after abc in all three arms (WB: recall 0.55 to 0.07), but the ground truth has 174 instances at S7 and 166 at S8 and the gate and LUT footprints are not like for like, so the comparison is not like for like; mux_bit falls by 0.189 and is read as not fooled by a margin of 0.011; for the other kinds the recall is already at a floor, and FSM (24 instances) and counter (18) are too few to say more. L15a POSSIBLY GOOD: under the random-set null the register, compare, mux and arithmetic kinds are each lost at one dominant pass (opt 320 of 580 losses, abc 166), but under the persistence null that placement is what a chance merging of the same objects also gives; the paper keeps only the descriptive statement (a compare is no longer a separable unit after the first opt, 18 of 19). L15c INCONCLUSIVE: the D14 labels explain 0.606 of the merged losses, the middle class of the rule.

H. What is recoverable in principle (H14a)

Hypothesis. H14a: the synthesis flow destroys style information, so that the pooled style-recovery ceiling at f3 is at most 0.60 (at least 40% of style variants are indistinguishable).

Experiment. The ceiling is measured on a twin set (the same design in every coding style) at f1-k6, f2 and f3. For each twin base design, with S styles and C distinct netlist classes at the floor, the style-recovery ceiling is C/S, the accuracy of the best possible guesser of the coding style from the netlist under a uniform prior (a Weisfeiler-Lehman class, confirmed by exact isomorphism where the netlist is small).

Result. The style-recovery ceiling is 0.470 at f1-k6, 0.482 at f2 and 0.500 at f3 (Table XXXIV).

Verdict. H14a USEFUL: the ceiling is 0.5000 at f3, so what any recovery can do is limited by what the flow destroyed, whatever the detector.

TABLE XXXIV
Style-recovery ceiling (classes of equivalent netlists over coding styles) on the twin set, per family and floor.

family	f1-k6	f2	f3
all	0.470	0.482	0.500
adder	0.667	0.667	0.667
alu	0.333	0.472	0.528
comparator	0.333	0.333	0.333
counter	0.333	0.333	0.333
dpfsm	0.500	0.500	0.500
fsm	0.500	0.500	0.500
mux	0.562	0.562	0.667
pipeline	0.333	0.333	0.333
regfile	0.500	0.500	0.500
register	0.333	0.333	0.333
shifter	0.625	0.625	0.625
subtractor	0.667	0.667	0.667

I. A synthetic test bed: the idiom corpus (H12a)

Hypothesis. H12a: a seeded corpus of synthetic idiom designs builds on every floor (at least 0.95 of the designs pass every equivalence check on every floor).

Experiment. The corpus generator has twelve idiom families, each parameterised by width, depth, encoding, reset and enable style and coding style, giving 1,200 designs split by parameter region: train values only (840), the same region with unseen text (120, iid), and at least one test-only value, such as an unseen width or coding style (240, ood). Odin III builds each on f1-k6, f2 and f3 and the three equivalence checks are run.

Result. 1,200 of 1,200 corpus designs pass every equivalence check on all three floors (Table XXXV); the smallest family has 100 designs. The corpus is a test bed and not a source of a lookup table: a signature dictionary mined from it wins on none of the seven eligible kinds on real designs (won=0/7 floor=f2 sets=micro,vetest), and a GNN trained on more corpus designs does not improve on real ones (slope -0.0026 per doubling, 95% interval $[-0.0097, 0.0094]$); both are DISMISSED (H12b, H12c).

Verdict. H12a USEFUL: the corpus builds (1200 of 1200). This is a validity check of the test bed and says nothing about transfer.

J. Repetition in big designs (H13a, H13b, H13c)

Hypotheses. H13a: big designs are mostly repeated structure (at least 0.50 of their words lie in a repeat group).

TABLE XXXV
The round-4 corpus: designs per family and split, and designs that build on f1-k6, f2 and f3 with every equivalence check passing.

family	train	iid test	ood test	ok on all floors
adder	70	10	20	100
alu	70	10	20	100
comparator	70	10	20	100
counter	70	10	20	100
dpfsm	70	10	20	100
fsm	70	10	20	100
mux	70	10	20	100
pipeline	70	10	20	100
regfile	70	10	20	100
register	70	10	20	100
shifter	70	10	20	100
subtractor	70	10	20	100

H13b: big designs repeat more than small ones (by at least 0.15). H13c: the repeated instances share the dictionary signature (at least 0.70 of the repeat groups).

Experiment. A word is a group of cells that act as one multi-bit signal; a repeat group is a set of at least two words of the same type and width whose cells induce the same subgraph (a Weisfeiler-Lehman hash of radius 2 over the cell types). The share of repeated structure is the share of the words with at least two cells and width at least two that lie in a repeat group. It is measured on the large designs of the VTR suite that Odin III builds at f3 (six designs: blob_merge, sha and stereovision0 to stereovision3; Table XXXVI) and, for H13b, on the small designs (the micro designs, the VerilogEval references and the training part of the corpus). For H13c the repeat groups of the big designs are checked at f2, where cells absorb their neighbours and the signatures can differ: the share of groups whose members have identical multisets of dictionary signatures.

Result. H13a: 6,653 of 7,167 words of the big designs lie in repeat groups, a share of 0.9283 (Table XXXVI gives each design: from 0.744 for blob_merge to 0.982 for stereovision2). H13c: of 911 repeat groups, 0.8606 have identical signature multisets. H13b: the big share is 0.2404 above the small share (0.9283 against 0.6879), above the 0.15 margin, but the result does not survive a change of the small baseline: against the micro designs and the VerilogEval references the difference is $+0.053$ (inconclusive by the rule), and against the VerilogEval references alone it is $+0.009$ (not supported).

Verdicts. H13a USEFUL: 0.9283 against the threshold 0.50, in all six designs (the smallest share is 0.744). H13c USEFUL: 0.8606 against the threshold 0.70. H13b POSSIBLY GOOD: it holds against the pooled small baseline that the rule fixed and flips with the choice of baseline, so it is stated only with that fragility and is not used as evidence on its own.

TABLE XXXVI
Big VTR designs at f3: cells, Odin III peak RSS (MB, all three floors), words with at least two cells and width ≥ 2, share of them in a repeat group, memories, buses of at least 8 bits, nets with at least 8 readers, FSM words.

design	cells	RSS	words	repeated	mem	buses	hi-fanout	fsm
blob_merge	31,778	4915.6	1,090	0.744	0	51	717	1
sha	10,629	841.7	225	0.924	0	29	99	0
stereovision0	56,208	1665.9	1,814	0.942	0	1263	182	0
stereovision1	142,932	1863.8	1,892	0.972	0	830	1430	48
stereovision2	254,419	4793.1	2,001	0.982	0	951	1724	0
stereovision3	859	75.8	145	0.834	0	9	10	1

TABLE XXXVII
Gate-Breaker metrics on the VerilogEval quartiles (one run, K=10, 4 workers, 180 s per call, 7,440 s wall clock, 740 calls; deltas vs the paper's O3-Mini in the text).

Quartile (n)	SR	S@1	S@5	S@10	AST	CFG
Q1 (36)	0.903	0.944	1.000	1.000	0.894	0.942
Q2 (37)	0.778	0.811	1.000	1.000	0.761	0.890

K. The language-model baseline: the Gate-Breaker replication (GB1, GB2)

Measurement. GB1 and GB2 are measurements, not hypotheses: they give the baseline that the language-model rows of this section (R2-L2, LLM1, LLM2) compare against. Gate-Breaker's metrics compare a recovered RTL against a reference: Success Rate, Success@k, AST similarity and CFG similarity (Section II). Loki recovers structure, not a new RTL text, so the direct comparison is the replication: the Gate-Breaker harness itself, run with the only language model available on the host.

Experiment. GB1 runs the harness on the simple benchmarks of the fallback micro set. GB2 runs it on the VerilogEval quartiles the original paper used (Table XXXVII): one run, $K = 10$, four workers, 180 s per call, 740 calls. The AST and CFG kernels are recomputed with common-width feature vectors (the first run padded per graph and truncated in the zip, which the v1.1.0 CFG probe found; those columns are not comparable to it).

Result. GB1: success rate 0.933 on Q1 and 0.829 on Q2, Success@10 1.000 in both quartiles, AST similarity 0.576 and 0.444, CFG similarity 1.000 and 0.736, with 169 of 174 extracted candidates functionally correct; this is a first replication on the fallback micro set, not the original paper's VerilogEval set. GB2 is in Table XXXVII.

This replication ran Q1 and Q2 only; the Loki-only results for Q3 and Q4 on the same designs are in Section V-O. Seventy-four of the 78 Q1+Q2 designs are usable (three excluded because their synthesized netlists fail the official testbench self-check, one because its 241 KB prompt exceeds the OS argument limit); every scored design is solved by attempt 5. The deltas against the paper's O3-Mini: Q1 success rate -0.092, AST -0.030, CFG -0.011; Q2 success rate -0.039, Success@5 +0.125, Success@10 +0.111, AST -0.164, CFG +0.083. The Q1 gap is harness no-output, not wrong RTL; the Q2 AST gap is truth-table and case references against the model's boolean expressions. Two caveats: the host's only model path routes through a gateway that aliases a DeepSeek backend as Claude Opus (so the model column is not verifiably the paper's comparison class), and the CLI exposes none of the paper's sampling flags (T=0.85, p=0.95). The CFG column uses the common-width node kernel (the odin3 CFG probe); the AST column uses the paper's pyverilog Algorithm 2 of Gate-Breaker.

Verdict. GB1 and GB2 MEASUREMENT: they carry no verdict and are not counted.

What the two lines of work measure is complementary. The language model recovers functional behaviour from the gates and names nothing reliably; Loki recovers the structures the flow preserved (hard blocks and the signal roles its features see) and proves what it claims. The Gate-Breaker future work – bus naming, datapath/control classification, partitioning – is exactly L2's table, and its numbers are the baseline a model-assisted Loki would have to beat.

L. Language model plus Loki's hints: a pilot (LLM1, LLM2)

Hypotheses. LLM1: a model given Loki's hints recovers RTL better than without them. LLM2: the netlist helps beyond what the model knows from the interface line alone.

Every other result in this paper uses no language model. This subsection reports one small pilot that does. The measure is the functional pass rate per arm, plus the refusal rate.

Setup. The pilot is a replica of the Gate-Breaker netlist-to-RTL task [1] on VerilogEval [10]. It uses 40 problems, 10 per complexity quartile, chosen by a fixed hash rank from the quartile files, and one call per problem per arm ($K = 1$). There are three arms. Arm A gives the gate netlist and the interface line. Arm B adds the claims that Loki makes from the flat netlist alone (no provenance names, no L0 ground truth). Arm C gives the interface line only and no netlist, as a memorisation control. All arms use the same two worked examples and the same answer format. Each call is a fresh claude-sonnet-5-5 subagent with no prior turns. The strict scorer compiles the reply with Icarus Verilog 12.0 against the reference testbench and requires exactly one line Mismatches: 0 in M samples; it also rejects replies that name the hidden reference module or print their own result line. A Gate-Breaker-equivalent count is reported beside it as a cross-check. A call with no reply file or an API refusal counts as a fail in the all-calls view.

Results. Table XXXVIII gives the pass rate per arm in both views, and Table XXXIX gives the exact two-sided McNemar tests on the paired set of 38 problems answered in all three arms. Arm B is nominally ahead of arm A in the paired view (35/38 against 33/38) and both are ahead of arm C (29/38), but no difference reaches $p < 0.05$: the

TABLE XXXVIII
LLM pilot, run re: pass rate per arm (strict scorer unless marked). $n = 40$ problems per arm, one draw each, claude-sonnet-5-5. Arm A: netlist; B: netlist plus Loki hints; C: interface only. GB-equiv.: Gate-Breaker-equivalent scorer.

View	A	B	C
All 40, refusal/no file = fail (strict)	35/40 (0.88)	35/40 (0.88)	30/40 (0.75)
All 40, refusal/no file = fail (GB-equiv.)	33/40 (0.82)	33/40 (0.82)	29/40 (0.72)
Answered only (strict)	35/40 (0.88)	35/38 (0.92)	30/40 (0.75)
Paired set, 38 (strict)	33/38 (0.87)	35/38 (0.92)	29/38 (0.76)
Paired set, 38 (GB-equiv.)	31/38 (0.82)	33/38 (0.87)	28/38 (0.74)

TABLE XXXIX
LLM pilot, run re: exact two-sided McNemar tests, paired set $n = 38$, strict scorer. "First only" counts problems the first arm passes and the second fails.

Comparison	Both	First only	Second only	Neither	p
B vs A	33	2	0	3	0.5000
A vs C	28	5	1	4	0.2188
B vs C	28	7	1	2	0.0703

smallest is B against C at $p = 0.0703$. This is a pilot at $n = 40$ with one draw per cell; it cannot support a claim about absolute rates or about the effect of the hints. In the all-calls view arms A and B are equal (35/40) because arm B's two missing reply files count as fails, and arm A passed both of those problems. Per-quartile cells have 8 to 10 problems each; in Q4 the all-calls rates are A 9/10, B 7/10, C 7/10, so the sign of the B-minus-A difference is not stable across quartiles (Section V-O gives every quartile with its tests and the power limit).

Refusals. The first complete prompt that we ran, the Gate-Breaker standard prompt, was stopped by the protocol's stop rule at 88 of 120 calls: 8 calls came back as API safeguard refusals (no model reply) and 7 as no-file calls (the subagent declined or answered in chat without writing the reply file). Under our own prompt re, which explains the research, the same 120 calls gave 0 API refusals and 2 no-file calls. Table XL gives the rates per arm. The two prompts differ in more than the refusal behaviour, so any comparison of pass rates between them is exploratory only. Refusal handling is part of the cost of using a general-purpose model for this task and has to be reported beside the pass rate.

Threats to validity. (i) The subagents were not tool-less. They had file and web tools loaded and the workspace instructions and a memory index in context, and the reference and test files sit on the same disk (decision 11, a blinding deviation that was accepted). The transcript audit of all 208 calls (standard 88, re 120) shows that no call used any tool other than reading its own call file and writing or editing its own reply file: no shell, search or web tool. This is evidence from behaviour, not a technical block. Two calls used more than the usual two tool calls (standard 073 and re 116); both touched only their own files and both are scored. (ii) Decision 12: the first batch used a step-by-step reasoning prompt (cot) that

TABLE XL
LLM pilot: API refusals and no-file calls per arm. standard stopped at order 88 of 120 (calls made); re ran all 120.

Run	Arm	Calls	API refused	No file	Total
standard	A	30	2	4	6/30 (0.20)
standard	B	29	3	1	4/29 (0.14)
standard	C	29	3	2	5/29 (0.17)
re	A	40	0	0	0/40 (0.00)
re	B	40	0	2	2/40 (0.05)
re	C	40	0	0	0/40 (0.00)

the API safeguard refused in 6 of 10 calls, even in arm C; those answers were discarded and the prompt replaced by standard. Decision 13: a refusal is never retried or reworded, and pass rates are reported with refusal as a fail and on the paired set. Decision 14: the re system prompt is ours, not Gate-Breaker's, so exact Gate-Breaker fidelity is dropped; the worked examples, user content and answer format are unchanged (120 of 120 user messages byte-equal). The re prompt mentions Loki hints in every arm, including arms A and C that carry none; this cue is the same in all arms. (iii) The prompts therefore differ from Gate-Breaker's. Sampling is one draw with no controlled temperature, and the 40-problem subset has a prompt-size cap that under-represents big designs, so these rates are not comparable with the published ones. The hints are uncalibrated at f3, and arm C controls for memorisation of VerilogEval; it is not a measure of recovery.

Verdicts. LLM1 INCONCLUSIVE: arm A 35/40 and arm B 35/40 (paired set of 38: A 33, B 35; exact McNemar $p = 0.5000$). LLM2 INCONCLUSIVE: arm C 30/40 (paired A against C $p = 0.2188$, B against C $p = 0.0703$). One draw per cell on 40 problems cannot support a claim about the hints in either direction.

M. Language model plus Loki's hints: the full run (LLM3, LLM4)

Hypotheses. LLM3: a model given Loki's hints recovers RTL better than the same model given the netlist alone. LLM4: the netlist helps beyond what the model knows from the interface line alone.

The pilot could not separate the arms, so the same question was asked on every eligible problem of the four quartile files. The plan (arms, scorer, primary outcome, tests, stop rules) was written before the runs and committed first.

Setup. 137 problems (144 in the quartile files, 7 excluded: four prompts over the 40000-byte cap, one reference that fails its own testbench, two whose hints cannot be built), arms A (netlist), B (netlist and Loki's hints) and C (interface only), three independent draws per problem and arm: 1233 calls to fresh claude-sonnet-5-5 subagents, with the pilot's prompts, scorer and answer format. The primary outcome is that a problem passes in at least 2 of its 3 draws. Per quartile, the exact two-sided McNemar

TABLE XLI
LLM full run: pass rates per arm, 137 problems, 3 draws, strict scorer. MAJ: passes in at least 2 of 3 draws.

Arm	Calls	pass@1	pass@3	MAJ
A netlist	411	0.888	0.942	123/137
B netlist + hints	411	0.878	0.942	124/137
C interface only	411	0.560	0.686	77/137

TABLE XLII
LLM full run: primary tests (MAJ), exact McNemar per quartile and stratified; discordant problems (first only, second only), Holm p over ten tests.

Comparison	Stratum	Discordant	p	Holm p
A vs B	Q1	0, 2	0.5000	1.0000
A vs B	Q2	1, 0	1.0000	1.0000
A vs B	Q3	0, 0	1.0000	1.0000
A vs B	Q4	2, 2	1.0000	1.0000
A vs B	all	3, 4	1.0000	1.0000
B vs C	Q1	19, 0	< 0.0001	< 0.0001
B vs C	Q2	12, 1	0.0034	0.0273
B vs C	Q3	9, 1	0.0215	0.1504
B vs C	Q4	12, 3	0.0352	0.2109
B vs C	all	52, 5	< 0.0001	< 0.0001

test; across quartiles, the exact stratified form (discordant problems pooled); Holm adjustment over the ten tests. A claim needs an adjusted $p < 0.05$.

Results. All 1233 calls were answered (0 API refusals, 0 no-file calls, 0 timeouts). Table XLI gives the pass rates and Table XLII the tests. Arms A and B are level (mean pass@1 0.888 against 0.878; A minus B +0.010, 95% interval $[-0.017, +0.036]$), with 3 problems only A passes and 4 only B passes. Arm C, with no netlist, still passes 0.560, so over half of the passes need only the interface line. The netlist adds the rest: B minus C is +0.319 ($[+0.241, +0.394]$), 52 problems only B passes against 5 only C passes (Holm $p < 0.0001$). By quartile the B-against-C difference passes the rule in Q1 and Q2; Q3 and Q4 have the same sign but Holm $p = 0.1504$ and 0.2109.

Threats to validity. (i) Dispatch: a Sonnet dispatcher subagent spawned the call subagents, so the controller session did not. (ii) Blinding: the call subagents were not tool-less. A probe lists Agent, Artifact, Bash, Edit, Read, Skill, ToolSearch, Write and the Docs tools, with the workspace instructions, a memory index and a terse-style hook text in context; the reference files are on the same disk. The transcript audit of all 1233 calls found 0 calls touching any path other than their own call file and reply file; 37 used more than two tool calls (paged reads, double writes, and in 3 calls a shell search of their own call file). All are scored as made. This is evidence from behaviour, not a technical block. (iii) The 7 excluded problems and the prompt-size cap under-represent the largest designs. (iv) Arm A is near the ceiling in Q1 and Q2, which limits how much any hint could show. (v) The system prompt mentions Loki hints in every arm, and sampling is not controlled.

Verdicts. LLM3 DISMISSED: A minus B +0.010 $[-0.017, +0.036]$, stratified discordant 3 against 4, Holm $p = 1.0000$; the interval rules out a gain larger than about 0.04. LLM4 USEFUL: B minus C +0.319, stratified 52 against 5, Holm $p < 0.0001$; per quartile Q1 and Q2 pass the rule and Q3 and Q4 are INCONCLUSIVE. Arm C passing 0.56 bounds what the netlist adds: B minus C = +0.319.

N. Language model plus Loki's hints, version 2 (LLM5, LLM6)

Hypotheses. LLM5: a model given Loki's linked hints (version 2) recovers RTL better than the netlist alone. LLM6: the model uses the hints, so hints with wrong content hurt.

The full run found no gain from the version-1 hints. Two defects were fixed before this run: the claims were weak (no counter kind, empty state-machine claims, duplicates), and the model was never told what the hints are. Version 2 writes one plain-English line per claim with a probability, links it into the netlist as a comment on the line of each cell the claim covers, and the prompt explains the hints and asks the model to name the claims it used. The plan was written and committed before the calls.

Setup. The 64 problems of Q3 (34) and Q4 (30), where arm A of the full run was not at the ceiling. Four arms: A2 (netlist only), B1' (the version-1 hints with the new prompt), B2 (version-2 hints) and W (version-2 claims moved to wires they do not describe; 5 problems whose wrong claims stayed too close to the true ones are left out of the W tests). Three draws per problem and arm: 768 calls to fresh claude-sonnet-5-5 subagents, the full-run scorer and tests (at least 2 of 3 draws; exact stratified McNemar; Holm over nine tests). The stop was set at 97 percent of the weekly meter (Peter, before the calls); the meter never passed 89.

Results. 762 calls produced a reply file; 6 did not (the model answered in chat), with 0 API refusals and 0 timeouts. Outcomes: pass 645, runtime 109, compile 8. Table XLIII gives the pass rates. In Q4 the model passes more often with the version-2 hints (pass@1 0.933 for B2 against 0.744 for A2, 0.800 for B1' and 0.789 for W); in Q3 all arms sit between 0.85 and 0.87. Over all 64 problems B2 minus A2 is +0.099 with 95% interval $[+0.052, +0.151]$ and B2 minus B1' is +0.073 $[+0.031, +0.115]$. Pass@3 is 0.906 for A2, B1' and B2 (W 0.875): the hints raise the consistency of a single draw, not the best-of-three ceiling. The primary test does not reach the rule: 4 problems only B2 passes against 0 only A2 passes, Holm $p = 1.0000$. A2 already passes 54 of 64 problems by that measure, so few problems can differ.

Use of the claims. The model names a claim in 107 of 192 B2 calls. FSM claims were used in 57 B2 calls and

TABLE XLIII
LLM hints v2: pass rates per arm, 64 problems, 3 draws, strict scorer. MAJ: passes in at least 2 of 3 draws.

Stratum	Arm	Problems	pass@1	pass@3	MAJ
Q3	A2	34	0.853	0.882	30/34
Q3	B1'	34	0.853	0.882	30/34
Q3	B2	34	0.873	0.882	30/34
Q3	W	34	0.863	0.882	30/34
Q4	A2	30	0.744	0.933	24/30
Q4	B1'	30	0.800	0.933	26/30
Q4	B2	30	0.933	0.933	28/30
Q4	W	30	0.789	0.867	25/30
All	A2	64	0.802	0.906	54/64
All	B1'	64	0.828	0.906	56/64
All	B2	64	0.901	0.906	58/64
All	W	64	0.828	0.875	55/64

all 57 passed; counter claims in 24 and all passed. In W, FSM claims were used in 33 calls with 27 passes, against 24 calls where an FSM claim was offered and not used with 19 passes (0.82 against 0.79, no difference); a used wrong state-machine hint passed less often than a used right one (27 of 33 against 57 of 57), but on different problems, so this is not a paired test. W minus A2 is $+0.026$ ($[-0.021, +0.073]$), with 2 problems only W passes and 2 only A2 passes.

Threats to validity. (i) Underpowered: the primary test counts discordant problems, A2 is near the ceiling, and 4 against 0 cannot pass after Holm; the pass@1 interval and the claim-use table are not the pre-stated rule. (ii) Blinding: as in the full run, the call subagents were not tool-less and saw the workspace instructions, a memory index and a terse-style hook text. The transcript audit of all 768 calls found 0 calls touching any path other than their own call file and reply file and 0 using other tools; 26 used more than two tool calls (paged reads, double writes, and 6 calls that edited their own reply file with Edit instead of Write). All are scored as made. (iii) Confound: A2 has neither the hint explanation nor the claim-use line, so B2 against A2 differs by both; B2 against B1' holds the prompt fixed and isolates the hint content. (iv) The probability of a sequential claim is a per-kind constant checked on the netlist, not a measure of designer intent, and it was not used in any analysis. The checks are in-sample. (v) Wrong claims (W) stay partly close to the true ones in small designs. (vi) Memorisation is not controlled in this run. Sampling is not controlled.

Verdicts. LLM5 INCONCLUSIVE: B2 against A2 stratified discordant 4 against 0, Holm $p = 1.0000$, although mean pass@1 is $+0.099$ $[+0.052, +0.151]$. The interval and the claim-use table suggest that version-2 hints help on Q4 and that the model reads and uses state-machine and counter claims; this is a pointer for a better-powered test, not a verdict. LLM6 DISMISSED: W against A2 discordant 2 against 2, Holm $p = 1.0000$; the hypothesis that wrong hints lower the pass rate is not supported by the pre-stated test.

O. Per-quartile results: Loki alone and Loki with a model (Q1, Q2)

Hypothesis and measurement. Q1: the hints help in some complexity quartiles (hypothesis; verdict INCONCLUSIVE, the cells hold 8 to 10 problems and no quartile has enough discordant pairs for $p < 0.05$). Q2: what Loki alone recovers per quartile (a measurement with no verdict).

Two questions ask the same thing per Gate-Breaker complexity quartile: is the model with Loki's hints better than the model without them, and what does Loki alone recover? All numbers here come from files already committed; no new run or model call was made for this subsection. Every table below uses one partition, the Gate-Breaker quartile files (41, 36, 36 and 31 problems), which are also the partition of the full language-model run. The 138 built designs that appear in a quartile file are binned by them; 12 built designs appear in no file and are left out. The paper's absolute complexity bins give a different partition, shown once in Table XLIX: of the 138 designs, 35 sit in a different quartile there (103 agree), and 9 of the 40 pilot designs would change quartile.

Loki with a model, per quartile. Table XLIV splits the pilot of Section V-L by quartile, and Table XLV gives the exact two-sided McNemar test in each quartile. Conditioning on the discordant pairs of each quartile, each discordant pair is equally likely to favour either arm under the null, so the sum of the first-only counts over quartiles is binomial over all discordant pairs: the stratified exact test equals the overall exact McNemar test, which is the All row of the table. Arm B is ahead of arm A by one problem in Q2 and in Q3, level in Q1, and behind by two in Q4 (all calls: 7/10 against 9/10). Both problems for which arm B has no reply file are Q4 problems, so in the paired Q4 view A and B are level at 7/8. Over all quartiles the paired difference is 2 problems for B and 0 for A ($p = 0.50$), and in the all-calls view 2 against 2 ($p = 1.0$). Arm C, which sees no netlist, is behind arm B in the point estimate in every quartile except Q4 (level 7/10), but the overall test of B against C is $p = 0.070$ in the paired view and $p = 0.18$ in the all-calls view, and no quartile reaches $p < 0.05$.

What the tests can show. A two-sided exact McNemar test needs at least 6 discordant pairs, all in one direction, for $p < 0.05$; with 8 discordant pairs it needs 8 to 0, and with 10 it needs 9 to 1. The largest number of discordant pairs in any single quartile comparison here is 3. Overall, B against C in the all-calls view has 9 discordant pairs split 7 to 2, where 8 to 1 would have been needed. These data therefore do not show that Loki's hints help in any quartile, and do not show that they do not. No claim beyond these tests is made. A larger run (all designs of the quartile files, 3 draws per arm) is planned and is what can test an effect.

TABLE XLIV
LLM pilot (run re) pass rate per Gate-Breaker complexity quartile (the quartile files), strict scorer. All calls: refusal or missing reply file counts as a fail. Paired: problems answered in all three arms. Arm A netlist, B netlist plus Loki hints, C interface only; one draw per cell. AST and CFG similarity were not scored in the pilot.

Quartile	View	n	A	B	C
Q1	all calls	10	10/10	10/10	8/10
Q1	paired	10	10/10	10/10	8/10
Q2	all calls	10	8/10	9/10	8/10
Q2	paired	10	8/10	9/10	8/10
Q3	all calls	10	8/10	9/10	7/10
Q3	paired	10	8/10	9/10	7/10
Q4	all calls	10	9/10	7/10	7/10
Q4	paired	8	7/8	7/8	6/8
All	all calls	40	35/40	35/40	30/40
All	paired	38	33/38	35/38	29/38

TABLE XLV
LLM pilot (run re): exact two-sided McNemar test per quartile, strict scorer. Cells are first-only/second-only discordant pairs (all-calls view in the first group, paired view in the second); p in brackets. The All row pools the discordant pairs of Q1–Q4; it is also the stratified exact test, because the sum of independent Binomial(d_i, 1/2) counts is Binomial($\sum d_i$, 1/2). A two-sided exact test needs at least 6 discordant pairs, all in one direction, for $p < 0.05$.

View	Stratum	B vs A	B vs C	A vs C
all calls	Q1	0/0 (1.000)	2/0 (0.500)	2/0 (0.500)
all calls	Q2	1/0 (1.000)	2/1 (1.000)	1/1 (1.000)
all calls	Q3	1/0 (1.000)	2/0 (0.500)	1/0 (1.000)
all calls	Q4	0/2 (0.500)	1/1 (1.000)	2/0 (0.500)
all calls	All (strat.)	2/2 (1.000)	7/2 (0.180)	6/1 (0.125)
paired	Q1	0/0 (1.000)	2/0 (0.500)	2/0 (0.500)
paired	Q2	1/0 (1.000)	2/1 (1.000)	1/1 (1.000)
paired	Q3	1/0 (1.000)	2/0 (0.500)	1/0 (1.000)
paired	Q4	0/0 (1.000)	1/0 (1.000)	1/0 (1.000)
paired	All (strat.)	2/0 (0.500)	7/1 (0.070)	5/1 (0.219)

Loki alone, per quartile. Table XLVI shows every Loki result that is defined on the VerilogEval designs, in the four quartiles. The L0 ceiling is flat: every node of every built design has a label row, a cell_type and an HDL span, and the module coverage is 1.000 in every quartile. Signal coverage has mean (median) 1.000 (1.000), 0.986 (1.000), 1.008 (1.000) and 0.896 (0.866) in Q1 to Q4. The 138 designs of the quartile files include the 6 references that are not built (3 in Q3, 3 in Q4: two that Odin III's SystemVerilog reader skips and four that fail Loki's own equivalence gate), so all three floors are built for 41, 36, 33 and 28 designs. The AST similarity of the re-emitted Verilog is lowest in Q2: calibrated f3 gives 0.691, 0.508, 0.602 and 0.533 for Q1 to Q4 on $n = 41$, 34, 30 and 15 designs, and the CFG similarity is 0.740, 0.750, 0.763 and 0.757 on $n = 41$, 36, 33 and 28. Designs whose reference pyverilog cannot parse have no AST value (0, 2, 3 and 13 in Q1 to Q4), so the Q3 and Q4 AST means rest on fewer, plainer designs and are not a clean effect of complexity. L1 changes little: calibrated and L0-stop f3 AST agree to the third decimal in Q1, Q2 and Q4 (0.602 against 0.612 in Q3), because the calibrated flow commits 14 hypotheses in 7 designs (round 2). The recovery experiment on the micro designs (Section V-B) is ranked by node count, not by Gate-Breaker quartile, and the loss audit of Section V-G reports pooled counts per unit kind; neither is defined per quartile, so neither appears in the tables.

Word recovery against HAL, per quartile. Table XLVII splits the round 5b comparison of Section V-F on the 28 test-split VerilogEval designs that are in a quartile file. Q1 and Q2 have no ground-truth word and every tool predicts none. In Q3 (5 designs) the three tools match 5, 5 and 6 of 7 ground-truth words; in Q4 (6 designs) Loki W matches 5 of 6 with 7 predicted words, Loki WB 4 of 6 with 11 and HAL 4 of 6 with 5. The counts are small, and they depend on the partition: with the absolute bins (Table XLIX) Q4 holds 10 designs and 139 ground-truth words.

Side by side. Table XLVIII puts the Loki-only AST and CFG, the round-1 Gate-Breaker harness replication and the pilot's pass rates in one table, with Loki and the round-1 model both on the common set (designs scored by both). The metrics differ: AST and CFG measure the similarity of recovered text, and the pilot measures functional correctness. On the common set Loki alone is below the round-1 model on AST and CFG in Q1 (AST 0.699 against 0.907, CFG 0.741 against 0.913) and Q2 (AST 0.512 against 0.733, CFG 0.754 against 0.917); the common set has one design in each of Q3 and Q4, so no mean is compared there (Section V-K). The pilot has no AST or CFG score, so the pilot and these columns are not compared on a common metric.

VI. Discussion and limits

Four observations follow from the verdicts, and then the candidate innovations.

First, the answer depends on what the flow erased. The structures that survived synthesis recover almost perfectly without any intelligence beyond reading the pass's own facts (L1h); the structures synthesis erased recover weakly (L1s) and, on the gate netlist, not at all (F3). Nothing in the experiments shows that the Bayesian machinery, as built, changes that: the evidence families add nothing over the per-detector priors (R2-L4), and a LUT6 re-map reaches the second floor's level only in-sample.

Second, the graph stage does not find the hierarchy, and the network result is possibly good. No clustering of the netlist graph finds the designer's modules well: NMI against the module path is at most 0.3220 (k-means), and the community methods score lowest (L2a, DISMISSED). The GNN beats 1-WL only when node, graphlet, WL and NPN features combine (L2b), and that is one run on one split.

Third, the provenance answer key is the part that works. It covers every gate node (L0b), the renamed-instance

TABLE XLVI
Loki without a language model on the VerilogEval references, per Gate-Breaker quartile (the quartile files; 12 built designs that are in no quartile file are left out). References: problems in the quartile file (41, 36, 36, 31). Built: designs with the floor built; the 6 references that are not built (3 in Q3, 3 in Q4) are 2 that Odin III's SystemVerilog reader skips (an explicit cast) and 4 that fail Loki's own equivalence gate, see the veval-l0 record. Coverage: per-design coverage on f3 (label rows, cell_type, HDL span, module scopes, declared signals), mean (median). Signal coverage exceeds 1 for ve_Prob080_timer (5 recovered against 4 declared; flattened array elements add names). AST and CFG: mean similarity of the re-emitted Verilog to the reference (round 2); n is the designs with a value (references that pyverilog cannot parse have no AST value), so the CFG mean can rest on fewer designs.

Measure	Q1	Q2	Q3	Q4
References in the quartile file	41	36	36	31
Built, f1-k6	41	36	33	28
Built, f2	41	36	33	28
Built, f3	41	36	33	28
Label coverage, f3	1.000 (1.000)	1.000 (1.000)	1.000 (1.000)	1.000 (1.000)
cell_type coverage, f3	1.000 (1.000)	1.000 (1.000)	1.000 (1.000)	1.000 (1.000)
HDL-span coverage, f3	1.000 (1.000)	1.000 (1.000)	1.000 (1.000)	1.000 (1.000)
Module coverage, f3	1.000 (1.000)	1.000 (1.000)	1.000 (1.000)	1.000 (1.000)
Signal coverage, f3	1.000 (1.000)	0.986 (1.000)	1.008 (1.000)	0.896 (0.866)
n (designs with an AST value)	41	34	30	15
n (designs with a CFG value)	41	36	33	28
L0-stop f3, AST	0.691	0.508	0.612	0.533
L0-stop f3, CFG	0.740	0.750	0.765	0.757
L1 calibrated f3, AST	0.691	0.508	0.602	0.533
L1 calibrated f3, CFG	0.740	0.750	0.763	0.757
L1 uncalibrated f3, AST	0.633	0.463	0.589	0.511
L1 uncalibrated f3, CFG	0.598	0.645	0.739	0.749
L0-stop f2, AST	0.734	0.527	0.609	0.541
L0-stop f2, CFG	0.828	0.840	0.837	0.856
L1 calibrated f2, AST	0.713	0.525	0.600	0.541
L1 calibrated f2, CFG	0.774	0.829	0.823	0.856
L1 uncalibrated f2, AST	0.652	0.497	0.591	0.510
L1 uncalibrated f2, CFG	0.649	0.787	0.797	0.811

TABLE XLVII
Round 5b on the VerilogEval test-split designs at f2 that are in a Gate-Breaker quartile file (28 of the 33), per quartile: word matches at Jaccard 0.5 over predicted words and over ground-truth words (matched/predicted/ground truth). Loki W: word-first detector; WB: with the block rule; HAL: DANA. Quartiles hold 5 to 10 designs, and designs without a ground truth word contribute zeros, so read these as counts, not rates.

Quartile	Designs	Loki W	Loki WB	HAL
Q1	10	0/0/0	0/0/0	0/0/0
Q2	7	0/0/0	0/0/0	0/0/0
Q3	5	5/6/7	5/7/7	6/6/7
Q4	6	5/7/6	4/11/6	4/5/6

protocol shows the pipeline is naming-neutral (L0a), and the same protocol exposed two defects (a rename-script defect that removed every hard proposal on renamed floors, which was fixed, and a spectral nondeterminism, which is reported). The loss audit (Section V-G) uses the same map: the mux word is the one kind that a pass (abc) demonstrably fools (L15d), and the placement of other losses at the first opt is what chance merging also gives (L15a, POSSIBLY GOOD).

Fourth, readable and verified are different properties. The language model produces RTL whose similarity to the reference is high and whose operators it names, but nothing proves the text equals the netlist; Loki's output is formally equal to its input in all runs and recovers few operators. The pilot (LLM1, LLM2) could not separate the arms; the full run (LLM3, LLM4) shows that the hints add nothing measurable to the netlist (LLM3 DISMISSED) while the netlist itself adds a lot (LLM4 USEFUL). A second run with better, linked hints (LLM5, LLM6) gave a Q4 pass@1 of 0.933 against 0.744 but did not meet the pre-stated test (LLM5 INCONCLUSIVE, LLM6 DISMISSED); it is a pointer, not a verdict.

Candidate innovations. Three ideas have only POSSIBLY GOOD support and stay out of the contributions. The graph network (L2b) needs several random splits with an interval over designs and a second benchmark suite. The operator-recovery F1 (R2-L7) needs a set with many more tie pairs and a flow that recovers operators, since the calibrated flow recovers none. The loss audit (L15a, L15d) needs equal-sized ground truths at the two netlist snapshots, so that the recalls are on the same instances, and a run on the big designs.

Limits. (i) The recovery stages use no language model; the pilot and the full run are separate experiments. (ii) Recovery is structural, not functional: the output is exactly as correct as its input netlist, which itself passes the official testbench in only some designs (inferred latches and asynchronous resets are the failures). (iii) Six rows are POSSIBLY GOOD and not USEFUL because they are narrow or fragile (L2b, R2-L7, H9a, H13b, L15a,

TABLE XLVIII

Per Gate-Breaker quartile, the Loki-only flow and the language-model arms side by side. Loki and Round-1 LLM are both taken on the common set, the designs scored by the round-1 Gate-Breaker harness replication (Table XXXVII) and by Loki, with the same AST and CFG metric; Loki is the calibrated f3 L1 output (round 2). n is the common designs with an AST value; n_{CFG} is the designs with a CFG value for Loki / the round-1 model. A dash is a quartile with no common design. Pilot arms: functional pass rate, all calls (strict scorer), n problems, a different metric that is not comparable with AST or CFG. Gate-Breaker's published numbers appear in the paper only as deltas (Section V-K), so no absolute published column is shown.

	Common set, Loki				Round-1 LLM		Pilot pass			
Quartile	n_{AST}	n_{CFG}	AST	CFG	AST	CFG	n	A	B	C
Q1	40 / 40	40 / 40	0.699	0.741	0.907	0.913	10	10/10	10/10	8/10
Q2	30 / 31	31 / 31	0.512	0.754	0.733	0.917	10	8/10	9/10	8/10
Q3	1 / 1	1 / 1	0.360	0.744	0.727	0.990	10	8/10	9/10	7/10
Q4	1 / 1	1 / 1	0.350	0.763	0.606	0.901	10	9/10	7/10	7/10

TABLE XLIX

Secondary view: the same Loki-only measures per quartile of the paper's absolute complexity bins (Table XXXVII), not of the Gate-Breaker quartile files. Of the 150 built designs, 12 are in no quartile file and 35 of the other 138 sit in a different quartile in the two partitions (cross-tabulation in partition_crosstab.csv). Cells as in Table XLVI; HAL cells are Loki W matched/predicted/ground truth. The Q4 signal-coverage mean includes ve_Prob153_gshare: 140 recovered signals against 15 declared (reference method: regex), a ratio of 9.33. The regex extractor does not undercount the source declarations (15 names: 11 ports, 3 internal nets, the array pht, counted once); the recovered count is consistent with the 128-entry array pht (2**n, $n = 7$) flattened into elements plus 12 other names, but the recovered signal list itself was not inspected.

Measure	Q1	Q2	Q3	Q4
Built designs, f3	38	39	37	36
Signal coverage, f3	0.987 (1.000)	1.000 (1.000)	0.974 (1.000)	1.161 (0.908)
AST n, L1 calibrated f3	38	38	32	21
AST, L1 calibrated f3	0.754	0.455	0.605	0.505
CFG n, L1 calibrated f3	38	38	36	32
CFG, L1 calibrated f3	0.750	0.741	0.750	0.768
HAL designs	11	7	5	10
Loki W, matched/pred/GT	0/0/0	0/0/0	5/6/7	8/142/139

L15d); round-1 claims (L0a to L2b) have a clear gap but no interval. (iv) The word-first detector, the signature dictionary and the synthetic-corpus GNN did not transfer from synthetic to real designs. (v) Area, speed and power are not measured, and the benchmarks are small designs: equivalence and recovery were not run on the big VTR designs.

VII. Future work

Four hypotheses are untested. Each is an idea and the experiment it needs.

L3, a cross-suite GNN. A GNN trained on one benchmark suite may transfer to the other. The experiment trains on the micro designs and tests on the VerilogEval designs and the reverse, with an interval over designs.

L6, a motif null model. The graphlet counts may differ from what a degree-preserving random graph gives. The experiment compares the ORCA graphlet counts of each netlist with those of random rewirings of the same graph.

L8, a model given the ReverseProvenance record. A language model that sees Loki's record may recover RTL better than the model alone. The experiment is the Gate-Breaker task with a record arm and a no-record arm on the full quartile files, three draws per arm. The full run of Section V-M did this for the hints (arm B against arm A) but not yet for the ReverseProvenance record.

Target-aware transformations. The flow that makes a netlist can recode a construct for its target: a state machine may be rewritten one-hot instead of binary, a sum may map to a carry chain, a memory to a hard block. Each changes the footprint the construct leaves in the netlist. The experiment (hub question L Q3.3) measures how each transformation changes the netlist footprint of a construct and whether Loki recovers more when it speculates which transformation was applied, starting with state-machine encoding and then carry chains and hard blocks. The hints-v2 run (LLM5) gives a first reason to try: state-machine claims are the ones the model used most.

For one-hot recoding, recall holds, but the claims are not selective. On the 100 generated state machines of round 4, built three ways (the fsm pass recoding one-hot, recoding binary, or leaving the HDL encoding) on the gate and the LUT floor, the version-2 claims of loki re –intent, which read how the state bits turn each other on, recover 98 of 100 one-hot machines and 99 of 100 binary ones, against 0.13 and 0.35 for the stage-one detector. The recall difference one-hot minus binary is -0.010 with 95% interval $[-0.025, 0.000]$, inside ± 0.10 (hypotheses FSM2a, FSM2b). The encoding call is right for 97 of 98 one-hot machines, but a binary machine whose state count is not a power of two is called other. The cost is precision: 446 of 577 counted claims sit on memory words of the 20 register-

file negatives, so the pooled precision is 0.17 against 0.70 to 0.92 (per design 0.82, and 0.75 without the register-file claims); FSM2a is therefore DISMISSED on its precision half, and FSM2b holds for recall only until the claims are selective.

A rule on the next-state function makes the claims more selective, but not enough. A memory word never decides where it goes next: for every control value its next state is its old state or one value whatever the state is (hold or load), and a counter with a load port can reach every code in one step. Rejecting these two patterns (chosen on the even-numbered half of the designs, scored on the odd-numbered half, fixed before the change) removes all 256 register-file claims per cell on the test half and loses no state machine (recall stays 1.00), and the pooled claim precision rises from 0.15 to 0.68 (per design 0.81 to 0.88). The bar of 0.80 is not reached: 21 claims on pipeline negatives and 3 on counters remain, and no tuning design produced one, so the rule has not seen them (hypothesis FSM3, DISMISSED).

a) Loki L3: design patterns.: The structures above are single constructs. Our observation is that the failures of the model baseline (Prob151_review2015_fsm, Prob156_review2015_fancytimer) are compositions: a sequence-detector state machine, a shift register that loads, a prescaler, a down counter and a done/ack handshake. We therefore introduce a third level, Loki L3: a catalogue of composite design patterns, each with its hardware forms, its composition from level-one claims, its netlist footprint per synthesis tool and per target technology, and a closed loop (inspect, hypothesise, rebuild the RTL, test against the netlist, refine) whose stages follow the loop described in a project note for a published software reverse-engineering system (inventory and parsing, evidence linked to exact file and line, explanation and proof, reconstruction and testing); we describe that system only from that note and make no comparison of results. The specification is docs/specs/2026-10-30-l3-patterns-design.md; a first detector (v0) is described below.

A first census (hypothesis register row P3C, a measurement) counts the seed patterns in 519 designs (the 156 VerilogEval references, 360 round-4 designs, three VTR designs), labelled from the register-transfer side, and synthesises each with Odin III (gates, LUT6, LUT6 with hard blocks) and Yosys (gates, LUT6). A state machine occurs in 78 designs, a counter in 44, a shift register in 11, a timer (counter with terminal compare) in 12, and a composite is rare: three designs have a state machine that drives a shift register and one has a state machine with counters and a shift register. The footprint changes with the target and the tool. On the hard-block target no counter claim survives (0 of 43; a hard-adder claim appears in all 43), against 37 of 43 on gates and LUTs. At the same gate target the terminal compare of a timer is a separate claim in 7 of 9 Yosys netlists and 2 of 11 Odin III netlists, and the Yosys gate netlists are about four times larger. Both tools recode state machines to one-hot, so the encoding call does not differ. The motivating fancy timer yields no state-machine, counter or shift-register claim in any Odin III cell, and the Gate-Breaker Yosys script turns it into constant outputs, which is the gap an L3 composite is meant to explain. No two claims of a composite were joined by a register-input path.

A first detector (v0; hypothesis register rows L3A and L3C, measurements with a stated rule) reads the level-one claims and the netlist through a JSON catalogue with one signature per target; on the hard-block target a counter is a hard-adder claim whose sum bits loop through flip-flops back to the adder with a one-bit or constant operand. On 252 held-out single designs it works for state machines (precision 0.85 to 0.95, recall 0.95), counters (no false alarm; recall 0.86 to 0.90 on gates and LUTs, 21 of 21 on the hard-block target) and glue logic; it does not work for shift registers (precision 0.14 to 0.17), timers (recall at most 0.17) or register files, and the LFSR has one held-out design. Most misses are a missing level-one claim, not a wrong composition. We then built 182 composite benchmarks by wiring 27 VerilogEval references (named by their problem numbers, for example Q035_146) with a known wiring; on the 82 held-out composites the parts are found 41 to 50% of the time and the wires between parts 12 to 18% (38 to 61% when both ends were found), below the bar of 0.70, so the combined benchmarks do not yet show that the composite idea works; they show which single claims must be repaired first. The Odin III and Yosys cells behave alike.

b) Loki L3: graphlet signatures and a library of every width.: Asked whether a partition's graphlet signature can say that it is a given pattern when exact matching cannot (the plan is in experiments/graphlets/EXPERIMENT.md), we built a library of 700 generated circuits (counters, shift registers, LFSRs, registers, timers, register files, state machines and glue, every width from 1 to 32; register files at four depths) in the five cells, all synthesised (3500 of 3500), and took their undirected (73 orbits) and directed (129 orbits) graphlet signatures with the Odin III graphlets pass; signatures of widths above 32 are made on demand and cached (tested at 33, 48 and 64 bits, 165 of 165 builds). The signature of a whole single design is a weak identifier: on 243 held-out single-pattern designs it does not reach precision and recall 0.80 for any pattern in any cell, and the rule detector of v0 stays better for counters, state machines and glue; a signature-or-rule hybrid recovers 39 of 60 register files (the rule alone, 6) with extra false alarms. Directed graphlets were more precise in one of 25 tests (state machines on Yosys gates) after Holm correction. Library circuits with a width left out are classified correctly 0.77 (undirected) and 0.80 (directed) of the time when only the flip-flop count is used and the nearest size stands in, and 0.95 to 1.00 when all other widths are searched. Cutting the design

into state groups (the registers the claims found, grown to the union of their input cones) and matching each cut with the library by its flip-flop count does better but not well enough: on single designs no pattern reaches 0.80 (one hybrid cell, counters on the hard-block target, does, where the rule already works); on the 82 held-out composites the signature alone finds 0.33 to 0.61 of the parts (rule 0.41 to 0.50) and, with the rule added, 0.52 to 0.74, with more false alarms (precision 0.25 to 0.36, rule 0.35 to 0.54). Directed graphlets were more precise for state machines on the two LUT cells and nowhere else (2 of 25). The Bayesian posterior stays below 0.3 for almost every partition, so it does not support a confident accept. Stopping each cone at the neighbouring registers, so that a partition holds one part and its own glue, raises the part precision of the signature alone on the composites from 0.23–0.35 to 0.35–0.87 and lowers its recall from 0.33–0.61 to 0.21–0.42; with the rule added, the parts are found 0.46–0.65 of the time (rule alone 0.41–0.50) at precision 0.32–0.57 (rule 0.35–0.54). Links between matched partitions recover the wires between parts at most 0.33 of the time, with precision 0.17–0.46 (rule: recall 0.12–0.18, precision 0.36–0.45), so they are not yet a usable wire detector. Partitions from the second-level clustering and a learned classifier are not yet tested.

VIII. Conclusion

Loki III asks how far netlist-to-RTL reverse engineering gets without a language model. We stated each claim as a hypothesis and gave it one experiment and one verdict. Eight hypotheses are USEFUL, seven are POSSIBLY GOOD with their fragility stated, twelve are DISMISSED and seventeen are INCONCLUSIVE. The structures that synthesis already reverse engineered are recovered (hard adders 67 of 67, with precision 0.985; hard multipliers 30 of 30), the stated probabilities are better calibrated though still far from exact (ECE 0.217 on the second floor, against 0.650 uncalibrated), and the pipeline is naming-neutral over 105,646 renamed identifiers. The structures that synthesis erased recover weakly, clustering does not find the designer's hierarchy, and a word-first detector is not shown to beat a published method. The part we would export to any future comparison, with or without a model, is the method: provenance-tracked, anonymised datasets with an answer key, plus the renamed-instance protocol. The graph network, the operator-recovery F1 and the loss audit are candidate innovations that await the experiments named in the Discussion. The full language-model run finds no effect of Loki's hints on a model that already has the netlist; the loss audit says where in the flow some constructs are lost, not yet how to recover them.

Appendix A
Research log

This appendix lists every hypothesis, one row each, including the DISMISSED and INCONCLUSIVE ones (Tables L to LIV); the rows are generated from docs/-paper/hypotheses.md. A MEASUREMENT row has no hypothesis and no verdict. UNTESTED rows are the Future work.

Sources. Every row's numbers are in a committed file, named by the Round column: R1 in experiments/rename-l2/EXPERIMENT.md, experiments/f3-recovery/EXPERIMENT.md and experiments/veval-l0/EXPERIMENT.md; R2 to R6 in experiments/round2 to experiments/round6 (each has an EXPERIMENT.md and the CSV files); HAL in experiments/hal and docs/results-hal.md; LLM in experiments/llm-pilot/EXPERIMENT.md; Q in experiments/quartiles. The exceptions are the success-rate and Success@k columns of the Gate-Breaker replication (rows GB1 and GB2), which come from run directories of the controller machine that are not committed; the AST and CFG columns of GB2 are in experiments/round2/l2_headtohead.csv.

Order of the work. Round 0 (2026-10-21 to 22) built L0 (provenance-labelled, anonymised datasets and the renamed-instance check), L1 (Bayesian hypotheses with calibration and proofs) and L2 (graph analysis). Round 1 (2026-10-23) gave the first results: hard blocks recovered, soft structures weak, nothing recovered on the gate netlist, the GNN ahead of 1-WL. Round 2 (2026-10-24) asked five follow-up questions (the LUT6 re-map, the operator F1, the head-to-head with the language model, the evidence families and the must/may bracket and structural pre-filter) and found that only the operator F1 is supported, weakly. Its puzzle, why wrong candidates are not real constructs, came from the author's question (2026-10-24): "for Loki - the question is from a reverse engineering perspective why is it hard to reconstruct to HDL - going from if structures (which we know are muxes) and then down it seems like many of the patterns are constant - so hypothesizing that things are a bus (and by a bus I mean a signal that could be a traditional bus, but also a wide wordsize signal that has regular oprtations to it) [...] Control flow is usually in the form of muxin and FSMs have pattern of feedback (state equals next state) and then control based on state. How do we find these patterns better?" and "Also, the outputs (words) and the inputs as words also provide hints of where the words are". Round 3 (2026-10-24) measured fragments and I/O propagation; round 4 (2026-10-25) built a synthetic corpus, a signature dictionary, a look at big designs and the identifiability ceiling; round 5 (2026-10-25) built the word-first detector; round 5b (2026-10-26) compared it with DANA; the LLM pilot and the per-quartile tables (2026-10-27 to 28) put a model beside Loki; round 6 (2026-10-27) is the loss audit.

Appendix B
The f3 experiment in detail

Hypothesis F3 (Section V-B): L1 recovers structure from Gate-Breaker's own gate input. Table LVII gives

TABLE L
Research log, hypotheses 1 to 12 of 73: every hypothesis, one row each, with the verdict.

ID	Round	Hypothesis	Result	Verdict
L0a	R1	The pipeline uses no names: renaming every identifier changes nothing.	all pairs byte-identical for L1 (204 + 204 + 68 sets); L2 clusters identical on 271 of 272 partitions (one spectral nondeterminism, also present on base)	useful
L0b	R1	The provenance answer key reconstructs the reference at every floor with no recovery step.	54,306 gate nodes all labelled; module coverage 1.000; signal coverage geomean 0.986; 6 of 156 not built	useful
L1h	R1	Structures the synthesis flow already reverse-engineered (hard adders and multipliers) are recovered almost perfectly.	hard adders 67/67 (P 0.985), hard multipliers 30/30	useful
L1s	R1	The Bayesian flow recovers the structures the flow erased.	f1 0.354 (271 counted, 107 matched, 688 GT); f2 0.108 (91/19/686); uncalibrated f1 0.202, f2 0.075	inconclusive
L1c	R1	Calibration makes the predicted probabilities honest.	calibrated 0.217 vs uncalibrated 0.650	useful
F3	R1	L1 recovers structure from Gate-Breaker's own gate input.	0 of 144 counted, 0 committed; buses proposed 2,569, none match; similarity probe: with the f3 calibration L1-stop equals L0-stop (every delta 0.0000); uncalibrated, 1,535 hypotheses in 11 of 15 designs move AST by +0.0111 and CFG by -0.0075; the flatten control raises AST by +0.0533 and CFG by +0.0559 (experiments/f3-recovery/EXPERIMENT.md)	dismissed
L2a	R1	Community methods find the designer's hierarchy; centroid methods do not.	NMI against module_path: k-means 0.3220, affinity propagation 0.2708, spectral 0.1378, Louvain 0.1360 (the reverse of the hypothesis); modularity is not a hierarchy measure (row L2m)	dismissed
L2m	R1	(measurement, no hypothesis) the modularity of the four clusterings of the netlist graph	Louvain 0.4220, spectral 0.3620, affinity propagation -0.2264, k-means -0.2912	measurement
L2b	R1	A GNN with node, graphlet, WL and NPN features beats the 1-WL baseline.	0.4241 vs 0.2717; accuracy 0.8863 vs 0.8492; node-only GNN loses (0.2413 vs 0.2711)	possibly good
R2-L1	R2	The f3 failure is detector vocabulary, not missing information.	0.87 in-sample (14 vs 16 constructs of 580), 0.51 held-out (2 vs 4)	inconclusive
R2-L2	R2	Loki's AST/CFG is below the LLM's on Q1/Q2 and does not fall Q1 to Q4, with output equal to input.	Loki AST 0.763/0.460 vs LLM 0.894/0.761 (Q1/Q2); AST falls Q1 to Q4 (0.754 ... 0.505); equivalence 150/150	inconclusive
R2-L4	R2	One or two evidence families carry the calibrated performance.	full 0.0038 vs priors-only 0.0051 (difference -0.0013; the gap is below the 0.02 floor)	inconclusive

the recovery per kind on the 15 f3 test designs, and Table LVIII gives the AST and CFG similarity per quartile of f3 node count.

References

[1] M. O. Faruque, P. Jamieson, A. Patooghy, and A.-H. A. Badawy, "Gate-breaker: An LLM-powered netlist-to-RTL reverse engineering tool," in Proc. IEEE Int. Performance, Computing, and Communications Conf. (IPCCC), 2025, pp. 363–370.

[2] P. A. Jamieson, "Improving the Area Efficiency of Heterogeneous FPGAs with Shadow Clusters," Ph.D. dissertation, University of Toronto, 2007.

[3] J. Luu, "Architecture-aware Packing and CAD Infrastructure for Field-Programmable Gate Arrays," Ph.D. dissertation, University of Toronto, 2014, tSpace handle 1807/68469.

[4] F. Gharibian, L. Shannon, and P. Jamieson, "Finding system-level information and analyzing its correlation to FPGA placement," in Proc. Int. Conf. Field Programmable Logic and Applications (FPL), 2010, pp. 544–549.

[5] Embedded Security Group, "HAL - the hardware analyzer," https://github.com/emsec/hal, 2019, mIT licence. Release v4.5.0 (2025-09-23).

[6] M. Fyrbiak, S. Wallat, P. Swierczynski, M. Hoffmann, S. Hoppach, M. Wilhelm, T. Weidlich, R. Tessier, and C. Paar, "HAL – the missing piece of the puzzle for hardware reverse engineering, trojan detection and insertion," IEEE Transactions on Dependable and Secure Computing, vol. 16, no. 3, pp. 498–510, 2019.

[7] N. Albartus, M. Hoffmann, S. Temme, L. Azriel, and C. Paar, "DANA: Universal dataflow analysis for gate-level netlist reverse engineering," IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES), 2020.

[8] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in Int. Conf. Learning Representations (ICLR), 2019.

[9] L. Alrahis, A. Sengupta, J. Knechtel, S. Patnaik, H. Saleh, B. Mohammad, M. Al-Qutayri, and O. Sinanoglu, "GNN-RE: Graph neural networks for reverse engineering of gate-level netlists," IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems, 2022.

[10] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "VerilogEval: Evaluating large language models for Verilog code generation," in Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD), 2023.

[11] W. Li, A. Gascon, P. Subramanyan, W. Y. Tan, A. Tiwari, S. Malik, N. Shankar, and S. A. Seshia, "WordRev: Finding word-level structures in a sea of bit-level gates," in 2013 IEEE International Symposium on Hardware-Oriented Security and Trust (HOST), Austin, TX, USA, Jun. 2013, pp. 67–74.

[12] P. Jamieson, K. B. Kent, F. Gharibian, and L. Shannon, "Odin II - An Open-source Verilog HDL Synthesis tool for CAD Research," in Proceedings of the IEEE Symposium on Field-Programmable Custom Computing Machines, 2010, pp. 149–156. [Online]. Available: http://www.computer.org/portal/web/csdl/doi/10.1109/FCCM.2010.31

[13] A. Mishchenko, S. Chatterjee, and R. K. Brayton, "Improvements to technology mapping for LUT-based FPGAs," IEEE Transactions on CAD, vol. 26, no. 2, pp. 240–253, 2007. [Online]. Available: http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.68.9003

TABLE LI
Research log, hypotheses 13 to 24 of 73: every hypothesis, one row each, with the verdict.

ID	Round	Hypothesis	Result	Verdict
R2-L5a	R2	Bracket width falls with per-design F1.	rho 0.087, p 0.417 (exploratory calibrated: -0.346, p 0.0008)	dismissed
R2-L5b	R2	A cheap structural rule prunes $\geq$ 30% of prover calls with no lost proof.	best zero-loss rule prunes 0.067	dismissed
R2-L7	R2	Word-level operator F1 separates runs that AST and CFG tie on.	tie pair dF1 0.0173; flatten control moves AST by 0.0422, operator F1 unchanged	possibly good
H9a	R3	Most false candidates are fragments of a word of a related kind.	pooled 0.583 (1,525 of 2,615); without the largest design 0.415, without ff_en 0.304 (exploratory); noise class unreachable by construction	possibly good
H9b	R3	A word's fragments cover most of it, so grouping recovers it.	0.065 (touched words only 0.069)	dismissed
H10a	R3	Datapath words are reached from the I/O words within two hops.	0.095 (per-design mean 0.326 exploratory)	dismissed
H10b	R3	Control words are not reached.	0.605 (needs $\leq$ D - 0.20)	dismissed
L11	R3	(measurement, no hypothesis) 0.214 of muxes with a select are chain sinks; complete chains 5 of 89 on VerilogEval		measurement
H12a	R4	The corpus builds on all floors (P_ok $\geq$ 0.95).	P_ok 1.0000 (1200/1200), min family 100	useful
H12b	R4	The dictionary beats the round-2 detectors on real designs.	won on 0 of 7 kinds (V0, V1, V2)	dismissed
H12b2	R4	The dictionary generalises over unseen regions.	R 0.3288 (iid 0.1866, ood 0.0614)	inconclusive
H12c	R4	GNN transfer to real designs improves with corpus size.	-0.0026, CI [-0.0097, 0.0094], 29 designs	dismissed

TABLE LII
Research log, hypotheses 25 to 36 of 73: every hypothesis, one row each, with the verdict.

ID	Round	Hypothesis	Result	Verdict
H13a	R4	Big designs are mostly repeated structure.	0.9283 (6,653 of 7,167)	useful
H13b	R4	Big designs repeat more than small.	0.2404 (0.9283 vs 0.6879); flips with the small baseline (experiments/round4/EXPERIMENT.md)	possibly good
H13c	R4	Repeated instances share the dictionary signature.	0.8606	useful
H14a	R4	The flow destroys style information.	0.5000 (f2 0.4823, f1 0.4697)	useful
H14b	R4	The coarser floor destroys more.	0.0177	inconclusive
H14c	R4	The MDL score ranks references above recovered RTL.	l0-f2 0.860 ... f3-none 0.940 (rule not met)	inconclusive
H14d	R4	The MDL score tracks round-2 similarity.	rho -0.5110, partial 0.3029	inconclusive
H16a	R5	Word tiling beats the detectors on held-out real designs.	q_WB 0.023 vs baseline 0.022, null 0.203	dismissed
H16b	R5	Grouping recovers the enabled words (R +0.20, F1 +0.10).	recall 0.008 to 0.146, F1 0.002 to 0.026, precision 0.014	inconclusive
H16c	R5	Word-level eq_const beats per-bit.	dF1 -0.000	dismissed
H16d	R5	The chain is complete more often.	WB 3 vs R2N 1	inconclusive
H16e	R5	Words grown from registers reach the datapath beyond a null.	D 0.829, null 0.715, gap 0.114	inconclusive

TABLE LIII
Research log, hypotheses 37 to 48 of 73: every hypothesis, one row each, with the verdict.

ID	Round	Hypothesis	Result	Verdict
H16f	R5	Repetition evidence raises F1 on big designs.	-0.0033; degenerate (margin unreachable, F1 bound 0.0165)	dismissed
RQ-H1	HAL	Loki's word-first detector recovers register words better than, equal to or worse than DANA.	Loki 0.169 vs HAL 0.261, D = -0.092, 95% CI [-0.317, 0.043]	inconclusive
GB1	R1	(measurement, no hypothesis) what the Gate-Breaker harness scores on the simple micro benchmarks with the model available on the host	success rate 0.933 (Q1) and 0.829 (Q2), Success@10 1.000 in both, AST 0.576 and 0.444, CFG 1.000 and 0.736; 169 of 174 candidates correct	measurement
GB2	R1	(measurement, no hypothesis) the baseline the model-based rows compare against	Q1 (36): SR 0.903, S@1 0.944, S@5 1.000, AST 0.894, CFG 0.942; Q2 (37): SR 0.778, S@1 0.811, S@5 1.000, AST 0.761, CFG 0.890	measurement
LLM1	LLM	A model given Loki's hints recovers RTL better than without.	A 35/40, B 35/40; paired 38: A 33, B 35 (B vs A discordant 2/0)	inconclusive
LLM2	LLM	The netlist helps beyond what the model knows from the interface line.	C 30/40; paired A 33 vs C 29	inconclusive
LLM3	Q	A model given Loki's hints recovers RTL better than without.	A 0.888, B 0.878; MAJ A 123/137, B 124/137; A-B +0.010 [-0.017, +0.036]; discordant 3 vs 4	dismissed
LLM4	Q	The netlist helps beyond what the model knows from the interface line.	C 0.560; B-C +0.319 [+0.241, +0.394]; MAJ B 124/137, C 77/137; discordant 52 vs 5	useful
LLM5	Q	A model given Loki's linked hints v2 recovers RTL better than the netlist alone.	B2 vs A2: pass@1 0.901 vs 0.802, +0.099 [+0.052, +0.151]; Q4 0.933 vs 0.744; MAJ B2 58/64, A2 54/64; discordant 4 vs 0	inconclusive
LLM6	Q	A model that uses the hints is hurt by hints with wrong content (W below A2).	W vs A2: pass@1 0.828 vs 0.802 (+0.026 [-0.021, +0.073]); discordant 2 vs 2; B2 names a claim in 107/192 calls; W FSM claims used 33, pass 27	dismissed
Q1	Q	The hints help in some quartiles.	B ahead by one in Q2 and Q3, level in Q1, behind by two in Q4	inconclusive
Q2	Q	(measurement) what Loki alone recovers per quartile	see quartile tables	measurement

TABLE LIV
Research log, hypotheses 49 to 60 of 73: every hypothesis, one row each, with the verdict.

ID	Round	Hypothesis	Result	Verdict
L15a	R6	Each construct kind is first lost at one dominant Odin III pass.	register, compare, mux, arithmetic dominant under the random-set null (opt 320, abc 166, flatten 68, second opt 26); under the persistence null (exploratory) dominant-null for all four (null shares 0.80, 0.84, 0.79, 0.46); only flatten of logic stays dominant	possibly good
L15c	R6	The D14 labels explain the merged losses.	0.606 pooled "partly explained"; MI 0.978, CI 0.462, VT 0.459; opt 0.425, abc 0.821	inconclusive
L15d	R6	A pass fools a detector.	mux word fooled by abc in all arms (WB 0.55 to 0.07; the ground truth is 174 instances at S7 and 166 at S8, so the two recalls are not on the same set); mux_bit: B and WB 0.201 to 0.012 (35 of 174 to 2 of 166), delta -0.189, which rule 4 reads as "not fooled" because it needs a delta of -0.20 or less; others at a floor	possibly good
L15b	R6	(descriptive) which constructs are joined with which, why		measurement
FSM2a	Q	The v2 FSM claims recover more FSMs than the stage 1 detector in every arm and floor without losing precision.	recall 0.98/0.99 against 0.13 to 0.51 (+0.48 to +0.85, Holm $p < 0.0001$); pooled precision 0.17 against 0.70 to 0.92 (446 of 577 counted claims on the 20 regfile negatives)	dismissed
FSM2b	Q	Reading how state bits turn each other on lets Loki recover one-hot FSMs about as well as binary ones.	-0.010 [-0.025, 0.000] pooled; encoding call right for 97 of 98 one-hot FSMs	possibly good
FSM3	Q	An FSM's next state depends on its own state bits through logic other than a hold/load mux; a rule that separates state machines from memory words keeps recall ≥ 0.95 and raises claim precision to ≥ 0.80. (controller's proposal, to be confirmed by Peter)	recall 1.000 (50 of 50, unchanged from v2); pooled claim precision 0.676 against 0.149 for v2 (regfile false claims 256 to 0; pipeline 21 and counter 3 remain); design precision 0.877	dismissed
P3C	Q	(measurement, no hypothesis) which composite patterns occur in the corpora and how their L1 claim footprint changes between Odin III and Yosys and between gate, LUT and hard-block targets	fsm 78 designs, counter 44, shift 11, timer 12, fsm_shift 3, fsm_counter 1, glue 199; counter claim 37/43 (Odin III gate, LUT), 38/41 (Yosys), 0/43 on hardmap (hard_add 43/43); timer compare claim 7/9 Yosys gate against 2/11 Odin III gate; FSM claim one-hot on both tools; 0 of 4 composites linked by a D path	measurement
L3A-fsm	Q	Does the L3 FSM detector find FSMs in single designs?	fsm TP 41 FP 7 FN 2 (O-gate) precision 0.85 recall 0.95; WORKING in O-lut, O-hard, Y-gate, Y-lut (precision 0.89 to 0.95, recall 0.95); O-gate INCONCLUSIVE (precision lower bound 0.77)	measurement
L3A-counter	Q	Does the L3 counter detector (hard-adder loop on the hard-block target) find counters?	counter FP 0 in all cells; recall 0.86 (O-gate, O-lut), 0.90 (Yosys), 1.0 (O-hard, 21/21); WORKING on O-hard, INCONCLUSIVE elsewhere (recall lower bound 0.71 to 0.78)	measurement
L3A-shift	Q	Do the L3 shift-register and LFSR detectors find them?	shift precision 0.14 to 0.17 (FP 12 to 17) recall 0.4 to 0.5: NOT WORKING in 5 of 5; lfsr 1 TEST design, no claim: INCONCLUSIVE	measurement
L3A-timer	Q	Does the L3 timer detector (counter plus terminal compare reaching the counter) find timers?	timer recall 0 to 0.17 (6 to 7 TEST timers), L1 compare claim absent in about half: NOT WORKING in 5 of 5	measurement

TABLE LV
Research log, hypotheses 61 to 72 of 73: every hypothesis, one row each, with the verdict.

ID	Round	Hypothesis	Result	Verdict
L3A-regfile	Q	Does the L3 register-file detector find register files?	regfile recall 0 to 0.31, precision 0 to 0.8: NOT WORKING in 5 of 5	measurement
L3A-register-glue	Q	Do the L3 plain-register and glue detectors work?	glue precision 0.98 to 1.0 recall 1.0: WORKING in 5 of 5; register recall 0.78 to 0.85: NOT WORKING in 3 Odin III cells, INCONCLUSIVE in 2 Yosys cells	measurement
L3C-composite	Q	Does the L3 composite detector find parts and wires in 82 TEST composites (Q-named benchmarks)?	part recall 0.41 to 0.50, wiring recall 0.12 to 0.18 (0.38 to 0.61 given both ends found), exact match 0 to 2 of 82: NOT WORKING in 5 of 5 (rule: wiring recall $\geq$ 0.70); 62 to 69 composites miss a part, 5 to 14 miss only a wire	measurement
GL1	Q	Graphlet signatures find the pattern kinds that the level-one claim chain misses.	whole-design signature alone: NOT WORKING in all patterns and cells (for example counter on gates, O-gate: precision 0.14 to 0.27, recall 0.43 to 0.86, the rule detector 1.00 and 1.00); signature OR rule recovers register files (TP 39 of 60 against 6) at the cost of false alarms; cones and composite partitions not yet tested	inconclusive
GL2	Q	Directed graphlets beat undirected ones in precision at equal or better recall.	directed more precise in 1 of 25 (fsm on Yosys gates: precision 0.78 against 0.50, recall 0.49 against 0.22, Holm p 0.0125); not in the other 24	inconclusive
GL3	Q	(measurement, no hypothesis) what a missing width costs: library without widths 5, 11, 19, 27 against those instances	undirected 339 of 440 (0.77), directed 352 of 440 (0.80) when the flip-flop count has no entry and the nearest size is used; 0.95 to 1.00 per pattern when all other widths are searched (undirected register 0.78)	measurement
GL4	Q	Partition signatures find the patterns the level-one claim chain misses.	singles: signature alone 0 of 40 rows WORKING; composites: part recall 0.33 to 0.61 (rule 0.41 to 0.50), 0.52 to 0.74 with the rule added, precision 0.23 to 0.36 (rule 0.35 to 0.54); neither reaches 0.70 on both	inconclusive
GL5	Q	Directed graphlets beat undirected ones in precision at equal or better recall on partitions.	directed more precise in 2 of 25 (state machines on O-lut: precision 0.33 to 0.73; on Y-lut: 0.13 to 0.64; Holm p $<$ 0.0001)	inconclusive
GL6	Q	A partition that holds one part plus its glue is matched more precisely, without a large loss of recall.	composites, count known: part precision 0.23 to 0.35 -> 0.35 to 0.87, recall 0.33 to 0.61 -> 0.21 to 0.42; with the rule added recall 0.46 to 0.65 (rule 0.41 to 0.50), precision 0.32 to 0.57 (rule 0.35 to 0.54); singles: signature alone still 0 of 40 rows WORKING, hybrid 4	inconclusive
GL7	Q	Links between matched partitions recover the wires of the composites.	rule recall 0.12 to 0.18; signature, all widths: recall 0.14 to 0.33 (precision 0.17 to 0.46); count known: recall 0.04 to 0.13	inconclusive
L3	-	A GNN trained on one suite transfers to the other suite.	not run; needs: Train on the micro designs, test on the VerilogEval designs and the reverse, with an interval over designs.	untested
L6	-	The graphlet counts differ from a motif null model.	not run; needs: Compare the ORCA graphlet counts of each netlist with degree-preserving random rewirings of the same graph.	untested

TABLE LVI
Research log, hypotheses 73 to 73 of 73: every hypothesis, one row each, with the verdict.

ID	Round	Hypothesis	Result	Verdict
L8	-	A language model given Loki's Reverse-Provenance record recovers RTL better than the model alone.	not run; needs: The Gate-Breaker task with a record arm and a no-record arm on the full quartile files, three draws per arm (deferred by Peter; the pilot of the LLM rows is the small first step).	untested

TABLE LVII

f3 L1 per kind on the 15 test designs. The calibrated columns are the protocol's setting; the –calib none columns are a diagnostic of what the detectors propose before calibration prunes it. A kind with no counted prediction has no F1 and is left out of the macro average.

		calibrated			–calib none			
Kind	GT	counted	matched	F1	counted	matched	precision	F1
adder	17	0	0	–	1	1	1.000	0.111
adder_cin	17	0	0	–	0	0	–	–
comparator	20	0	0	–	14	0	0.000	0.000
eq_const	16	0	0	–	1,141	0	0.000	0.000
ff_srst	2	0	0	–	7	0	0.000	0.000
hard_add	17	0	0	–	0	0	–	–
hard_mul	11	0	0	–	0	0	–	–
mux	7	0	0	–	0	0	–	–
mux_bit	7	0	0	–	0	0	–	–
register	22	0	0	–	3	1	0.333	0.080
subtractor	8	0	0	–	0	0	–	–
macro-F1	144	0	0	0.000	1,166	2		0.038
buses	120	2,569	0	0.000	2,835	0	0.000	0.000
datapath role	20,541	20,541	20,541	1.000	20,541	20,541	1.000	1.000
control role	0	0	0	–	0	0	–	–

TABLE LVIII

f3 AST (Gate-Breaker Algorithm 2, depth 3, pyverilog) and CFG (Algorithm 3, odin3 probe) similarity against the micro reference RTL, L0-stop (gate netlist) against L1-stop (design.v); quartiles by f3 node count. With the f3 calibration L1-stop equals L0-stop, so every delta is 0.0000; the table therefore shows the L0 values and the diagnostic (–calib none) deltas.

Quartile	AST L0	AST L1	ΔAST	CFG L0	CFG L1	ΔCFG
0 (4 designs)	0.4428	0.4428	0.0000	0.7909	0.7909	0.0000
1 (4)	0.3765	0.3765	0.0000	0.4224	0.4224	0.0000
2 (4)	0.3969	0.3969	0.0000	0.7013	0.7013	0.0000
3 (3 AST / 1 CFG)	0.4003	0.4003	0.0000	0.7507	0.7507	0.0000
mean (15 / 13)	0.4044	0.4044	0.0000	0.6468	0.6468	0.0000
diagnostic –calib none (L1 commits hierarchy)						
0 (4)	0.4428	0.4277	−0.0151	0.7909	0.7776	−0.0133
1 (4)	0.3765	0.3756	−0.0009	0.4224	0.4099	−0.0124
2 (4)	0.3969	0.4542	+0.0573	0.7013	0.7014	+0.0001
3 (3 AST / 1 CFG)	0.4003	0.4007	+0.0004	0.7507	0.7561	+0.0055
mean	0.4044	0.4155	+0.0111	0.6468	0.6394	−0.0075